

DevOpsShack

150 Common Errors · Root Cause Analysis · Solutions · Examples

Edition 2026 | devopsshack.com

Introduction

Welcome to the DevOpsHack Terraform Error Bible — a comprehensive reference guide covering 150 of the most frequently encountered Terraform errors, complete with Root Cause Analysis (RCA), actionable solutions, and real-world code examples.

Terraform has become the de-facto standard for infrastructure as code across cloud providers. As configurations grow in complexity — spanning multiple providers, modules, workspaces, and CI/CD pipelines — errors naturally arise. This guide equips engineers at all experience levels to diagnose and resolve these errors quickly.

Each error entry follows a consistent structure:

- Error Message — The exact error text or pattern as it appears in Terraform output
- Root Cause Analysis (RCA) — Why the error occurs and what triggers it
- Solution — Step-by-step guidance to resolve the issue
- Example — Practical HCL code demonstrating the fix

Errors are organized into thematic categories: Authentication & Provider, State Management, Syntax & Configuration, Planning & Apply, Resource-Specific, Modules, Workspace & Environment, Security, and CI/CD automation. Use the error number (#001–#150) to quickly reference specific issues.

Authentication & Provider

Errors #001–#005

#001 Error: No valid credential sources found for AWS Provider

Root Cause Analysis (RCA)

Terraform cannot locate valid AWS credentials. This occurs when `AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY` environment variables are missing, the `~/.aws/credentials` file is absent or malformed, or an IAM role isn't attached to the execution environment.

Solution

Configure credentials via environment variables, AWS credentials file, or attach an appropriate IAM role. Never hardcode credentials in .tf files. Use `aws configure` or set environment variables before running terraform plan/apply.

Example

```
# Option 1: Environment variables
export AWS_ACCESS_KEY_ID="AKIAIOSFODNN7EXAMPLE"
export AWS_SECRET_ACCESS_KEY="wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY"

# Option 2: Provider block with profile
provider "aws" {
  region = "us-east-1"
  profile = "my-profile"
}
```

#002 Error: Failed to query available provider packages

Root Cause Analysis (RCA)

Terraform cannot reach the `registry.terraform.io` endpoint, often due to network restrictions, firewall rules, proxy misconfiguration, or air-gapped environments. DNS resolution failures also trigger this error.

Solution

Check network connectivity to `registry.terraform.io`. Configure an HTTP proxy using `HTTPS_PROXY` environment variable or set up a private registry mirror. For air-gapped setups, use `terraform providers mirror`.

Example

```
# Set proxy for terraform
export HTTPS_PROXY="http://proxy.company.com:8080"
export HTTP_PROXY="http://proxy.company.com:8080"

# Mirror providers locally
terraform providers mirror /path/to/local/mirror

# Reference local mirror in .terraformrc
provider_installation {
  filesystem_mirror {
    path = "/path/to/local/mirror"
  }
}
```

```
include = ["registry.terraform.io/*/*"]
}
```

#003 Error: Invalid provider configuration – provider 'aws' requires explicit configuration

Root Cause Analysis (RCA)

A resource or module references an AWS provider that hasn't been configured in the root module or passed via the providers argument. This frequently occurs in multi-provider or multi-region setups.

Solution

Define all required provider configurations explicitly. When using provider aliases for multi-region deployments, pass the correct alias using the providers argument in module blocks.

Example

```
provider "aws" {
  alias = "us_east"
  region = "us-east-1"
}
provider "aws" {
  alias = "eu_west"
  region = "eu-west-1"
}
module "eu_resources" {
  source = "../modules/networking"
  providers = {
    aws = aws.eu_west
  }
}
```

#004 Error: Provider configuration not present

Root Cause Analysis (RCA)

The terraform state references a provider that no longer has a configuration block in the current .tf files. Happens after removing a provider block while the state still contains resources managed by it.

Solution

Either restore the provider configuration block, or use terraform state rm to remove orphaned resources from state before destroying them manually through the cloud console.

Example

```
# Check what resources use the provider
terraform state list | grep "aws_"

# Remove a resource from state if provider is gone
terraform state rm aws_instance.legacy_server

# Or restore the provider block temporarily
provider "aws" {
  region = "us-west-2"
}
```

#005 Error: Could not load plugin – incompatible API version

Root Cause Analysis (RCA)

The Terraform binary version and the provider plugin version are incompatible. This often happens after upgrading Terraform CLI without updating provider versions, or when a cached plugin in `.terraform/plugins` is outdated.

Solution

Delete the `.terraform` directory and `.terraform.lock.hcl` file, update provider version constraints in `required_providers` block, then run `terraform init` to download compatible provider versions.

Example

```
# Clean plugin cache
rm -rf .terraform .terraform.lock.hcl

# Specify compatible version constraints
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
  required_version = ">= 1.5.0"
}

# Re-initialize
terraform init
```

State Management

Errors #006–#010

#006 Error: Failed to load state: state data in S3 does not have the expected content

Root Cause Analysis (RCA)

The S3 bucket contains a corrupted or empty state file. This can result from incomplete writes during a previous apply, manual edits to the state file, or concurrent writes without proper locking.

Solution

Restore the state from an S3 versioned backup. Always enable versioning on your state bucket. Use terraform state pull to review the current state and terraform state push to restore a good version.

Example

```
# Enable S3 versioning (run once via AWS CLI)
aws s3api put-bucket-versioning \
  --bucket my-tf-state-bucket \
  --versioning-configuration Status=Enabled

# List object versions to find a good one
aws s3api list-object-versions \
  --bucket my-tf-state-bucket \
  --prefix terraform.tfstate

# Restore a specific version
aws s3api get-object \
  --bucket my-tf-state-bucket \
  --key terraform.tfstate \
  --version-id <VERSION_ID> \
  restored.tfstate
```

#007 Error: Error locking state: Error acquiring the state lock

Root Cause Analysis (RCA)

Another Terraform process is running and holds the DynamoDB state lock, or a previous run crashed and left a stale lock entry. Running concurrent terraform operations causes this.

Solution

Wait for the other process to complete, or manually remove the stale lock using terraform force-unlock with the lock ID shown in the error message. Investigate why the previous run didn't release the lock.

Example

```
# Get lock ID from error message, then force unlock
terraform force-unlock LOCK_ID_FROM_ERROR

# Verify lock table structure
aws dynamodb scan \
  --table-name terraform-state-locks \
  --filter-expression "attribute_exists(LockID)"
```

```
# backend config with locking
terraform {
  backend "s3" {
    bucket      = "my-tf-state"
    key         = "prod/terraform.tfstate"
    region      = "us-east-1"
    dynamodb_table = "terraform-state-locks"
    encrypt     = true
  }
}
```

#008 Error: Saved plan is stale – the state has been modified since this plan was saved

Root Cause Analysis (RCA)

The infrastructure state changed between when terraform plan -out=plan.tfplan was run and when terraform apply plan.tfplan was executed. Another engineer or pipeline applied changes in the interim.

Solution

Re-run terraform plan to generate a fresh plan file and apply it immediately. Implement a CI/CD pipeline that runs plan and apply in the same atomic job to minimize the window for staleness.

Example

```
# Regenerate the plan
terraform plan -out=fresh_plan.tfplan

# Apply immediately
terraform apply fresh_plan.tfplan

# In CI/CD, chain the steps
steps:
  - name: Terraform Plan
    run: terraform plan -out=tfplan
  - name: Terraform Apply
    run: terraform apply -auto-approve tfplan
```

#009 Error: Backend initialization required, please run 'terraform init'

Root Cause Analysis (RCA)

The backend configuration has changed (e.g., moved from local to S3) or the .terraform directory is missing. Terraform detects that its internal state doesn't match the backend configuration.

Solution

Run terraform init to initialize or re-initialize the backend. Use -migrate-state flag when changing backends to move existing state to the new location.

Example

```
# Initialize with backend config
terraform init

# Migrate state from local to remote backend
terraform init -migrate-state

# Reconfigure backend (skip migration prompt)
terraform init -reconfigure
```

```
# Provide backend config at init time
terraform init \
  -backend-config="bucket=my-state-bucket" \
  -backend-config="key=prod/terraform.tfstate" \
  -backend-config="region=us-east-1"
```

#010 Error: state snapshot was created by Terraform v1.x.x, which is newer than current v1.y.y

Root Cause Analysis (RCA)

Someone applied changes using a newer version of Terraform, and the state file was written with a version marker that the current (older) Terraform binary cannot read.

Solution

Upgrade the Terraform CLI to at least the version that wrote the state. Establish a team-wide version management policy using tfenv or .terraform-version files to ensure everyone uses the same version.

Example

```
# Install tfenv to manage versions
brew install tfenv # macOS
tfenv install 1.7.0
tfenv use 1.7.0

# Create .terraform-version file in project root
echo "1.7.0" > .terraform-version

# Verify active version
terraform version
```


Syntax & Configuration

Errors #011–#020

#011 Error: Unsupported argument – An argument named 'X' is not expected here

Root Cause Analysis (RCA)

The HCL configuration block contains an attribute name that doesn't exist in the provider's schema for that resource. Often caused by typos, deprecated argument names, or using documentation from a different provider version.

Solution

Check the provider documentation for the exact argument names supported by the current provider version. Use terraform providers schema to inspect available attributes, or check the CHANGELOG for renamed arguments.

Example

```
# Wrong: 'instance_class' used instead of 'instance_type'
resource "aws_instance" "bad" {
  instance_class = "t3.micro" # WRONG
}

# Correct
resource "aws_instance" "good" {
  ami      = "ami-0c55b159cbf1f0"
  instance_type = "t3.micro" # CORRECT
}

# Inspect available schema
terraform providers schema -json | jq '.provider_schemas'
```

#012 Error: Invalid block definition – A block definition must have block content delimited by '{' and '}'

Root Cause Analysis (RCA)

HCL syntax is malformed. Common causes include missing closing braces, extra parentheses, incorrect nesting of blocks, or copy-paste errors that introduced non-HCL syntax.

Solution

Use terraform validate to pinpoint the exact line. Ensure all blocks are properly opened and closed. Use a code editor with HCL syntax highlighting (VS Code + HashiCorp extension) to catch syntax errors early.

Example

```
# Wrong – missing closing brace
resource "aws_security_group" "bad" {
  ingress {
    from_port = 80
    to_port   = 80
  }
  # Missing closing brace for ingress block!
}

# Correct
resource "aws_security_group" "good" {
  name = "web-sg"
```

```
ingress {  
  from_port = 80  
  to_port   = 80  
  protocol  = "tcp"  
  cidr_blocks = ["0.0.0.0/0"]  
}
```

#013 Error: Variables not allowed – Variables may not be used here

Root Cause Analysis (RCA)

A variable reference is used in a location where Terraform evaluates configuration before variables are known, such as inside backend configuration blocks, module source arguments, or provider version constraints.

Solution

Replace variable references in backend blocks with literal values or use `-backend-config` flags at init time. For module sources, use literal strings. Extract dynamic values to wrapper scripts that call terraform with appropriate flags.

Example

```
# Wrong – variables in backend  
terraform {  
  backend "s3" {  
    bucket = var.state_bucket # NOT ALLOWED  
  }  
}  
  
# Correct – literal values in backend  
terraform {  
  backend "s3" {  
    bucket = "my-literal-bucket-name"  
    key    = "prod/terraform.tfstate"  
    region = "us-east-1"  
  }  
}  
  
# Or pass via -backend-config at init  
terraform init -backend-config="bucket=my-bucket"
```

#014 Error: Argument or block definition required – An argument or block definition is required here

Root Cause Analysis (RCA)

A required argument is missing from a resource or data source block. The provider schema mandates certain attributes that haven't been provided in the configuration.

Solution

Consult the provider documentation for the resource's required arguments. Run `terraform validate` to see the full list of missing required attributes. Add all required fields to the resource block.

Example

```
# Wrong – missing required ami and instance_type  
resource "aws_instance" "incomplete" {  
  tags = { Name = "my-server" }  
}  
  
# Correct – all required fields provided
```

```
resource "aws_instance" "complete" {
  ami      = "ami-0c55b159cbfafa1f0"
  instance_type = "t3.micro"
  tags = {
    Name      = "my-server"
    Environment = "production"
  }
}
```

#015 Error: Invalid function call – A call to function 'X' failed

Root Cause Analysis (RCA)

A built-in Terraform function was called with invalid arguments—wrong type, wrong number of arguments, or an out-of-range index. Examples include `tostring()` on a map, `element()` with an out-of-bounds index, or `jsonencode()` on unsupported types.

Solution

Verify the function signature in the Terraform documentation. Use terraform console to interactively test function calls before embedding them in configurations. Add type constraints and length checks where needed.

Example

```
# Test in terraform console
$ terraform console
> length(["a","b","c"])
3
> element(["a","b","c"], 5) # Will error if strict
> try(element(["a","b","c"], 5), "default")
"default"

# Safe pattern with try()
locals {
  safe_value = try(
    element(var.availability_zones, var.az_index),
    "us-east-1a"
  )
}
```

#016 Error: Invalid template interpolation value – cannot convert type 'list' to 'string'

Root Cause Analysis (RCA)

A list or complex type is used directly in a string template interpolation where Terraform expects a scalar string. Lists and maps cannot be converted to strings implicitly.

Solution

Use `join()` to convert lists to strings, `jsonencode()` for complex types, or `tostring()` for simple type conversions. Access specific list elements using index notation.

Example

```
# Wrong – list in string interpolation
variable "cidrs" { default = ["10.0.0.0/8", "192.168.0.0/16"] }
output "bad" { value = "CIDRs: ${var.cidrs}" }

# Correct options
output "joined" { value = "CIDRs: ${join(" ", var.cidrs)}" }
```

```
output "json" { value = "CIDRs: ${jsonencode(var.cidrs)}" }
output "first" { value = "First CIDR: ${var.cidrs[0]}" }
```

#017 Error: Reference to undeclared resource – A managed resource 'X.Y' has not been declared

Root Cause Analysis (RCA)

A resource reference like `aws_instance.web` is used but no such resource block exists in the configuration. Common causes include typos in the resource name, the resource being in a different module, or the resource having been deleted.

Solution

Verify the resource type and name exactly match a declared resource block. Check if the resource is in a module and reference it with `module.name.output`. Use `terraform state list` to see available resources.

Example

```
# Wrong – referencing non-existent resource
resource "aws_security_group_rule" "bad" {
  source_security_group_id = aws_security_group.web_servers.id # typo?
}

# Correct – exact name match
resource "aws_security_group" "web" {
  name = "web-sg"
}

resource "aws_security_group_rule" "allow_http" {
  type            = "ingress"
  from_port       = 80
  to_port         = 80
  protocol        = "tcp"
  source_security_group_id = aws_security_group.web.id # correct
  security_group_id      = aws_security_group.web.id
}
```

#018 Error: Invalid count argument – The 'count' object cannot be used in this context

Root Cause Analysis (RCA)

The `count.index` meta-argument is referenced outside of a resource that has a `count` attribute set, or it's used in a location like module arguments where `count` hasn't been applied.

Solution

Ensure `count` is set on the resource or module, and only use `count.index` within expressions that are within the scope of that counted resource. For modules, pass the index as an input variable.

Example

```
# Correct use of count.index
resource "aws_instance" "web" {
  count    = 3
  ami     = "ami-0c55b159cbfafa1f0"
  instance_type = "t3.micro"
  tags = {
    Name = "web-server-${count.index + 1}"
  }
}

# Access specific instance
output "first_ip" {
```

```
value = aws_instance.web[0].private_ip
}

# All IPs
output "all_ips" {
  value = aws_instance.web[*].private_ip
}
```

#019 Error: Invalid for_each argument – The given value is not suitable for for_each

Root Cause Analysis (RCA)

The value passed to `for_each` is not a map or set of strings. Lists, numbers, null values, or computed values that aren't known until apply time cannot be used with `for_each`.

Solution

Convert lists to sets using `toset()`, or create a map with meaningful keys. If the collection depends on data sources, use `depends_on` or `restructure` to break the dependency cycle.

Example

```
# Wrong – list passed to for_each
resource "aws_iam_user" "bad" {
  for_each = ["alice", "bob"] # list not allowed
  name     = each.key
}

# Correct – toset() converts list to set
resource "aws_iam_user" "good" {
  for_each = toset(["alice", "bob", "charlie"])
  name     = each.key
}

# Using a map for richer iteration
resource "aws_iam_user" "team" {
  for_each = {
    alice = "DevOps"
    bob   = "Developer"
  }
  name = each.key
  tags = { Team = each.value }
}
```

#020 Error: Invalid depends_on reference – References in depends_on must be to resources or modules only

Root Cause Analysis (RCA)

The `depends_on` argument contains a reference to a variable, output, or expression instead of a direct resource or module reference. Only resource addresses and module addresses are valid in `depends_on`.

Solution

Use only resource or module references in `depends_on`. If you need to create a dependency on a variable-based value, refactor the configuration to use explicit resource references.

Example

```
# Wrong – variable in depends_on
resource "aws_instance" "bad" {
  ami     = "ami-xxx"
  depends_on = var.some_var
}
```

```
instance_type = "t3.micro"
depends_on    = [var.some_variable] # WRONG
}

# Correct – resource reference only
resource "aws_instance" "good" {
  ami      = "ami-0c55b159cbfafa1f0"
  instance_type = "t3.micro"
  depends_on = [aws_security_group.web] # CORRECT
}
```

Planning & Apply

Errors #021–#025

#021 Error: Error creating instance: InvalidParameterValue – Invalid value for InstanceType

Root Cause Analysis (RCA)

The specified instance type doesn't exist in the target region or availability zone. Not all AWS instance families are available in all regions, and some require opt-in.

Solution

Check instance type availability using AWS CLI or console for the specific region. Use data sources to query available instance types dynamically, or select a widely available type like t3.micro.

Example

```
# Check available instance types in a region
aws ec2 describe-instance-type-offerings \
  --location-type "region" \
  --filters "Name=instance-type,Values=t4g.*" \
  --region us-east-1

# Dynamically validate instance type with data source
data "aws_ec2_instance_type" "selected" {
  instance_type = var.instance_type
}

resource "aws_instance" "web" {
  ami      = data.aws_ami.latest.id
  instance_type = data.aws_ec2_instance_type.selected.id
}
```

#022 Error: Error applying plan: timeout while waiting for state to become 'active'

Root Cause Analysis (RCA)

A resource is taking longer to reach the expected state than Terraform's default timeout allows. This is common for RDS instances, EKS clusters, large EC2 instances, and complex networking resources.

Solution

Add a timeouts block to the resource with extended values. The exact timeout keys (create, update, delete) vary by resource—consult provider documentation for available timeout options.

Example

```
resource "aws_db_instance" "main" {
  identifier      = "prod-db"
  engine          = "postgres"
  engine_version  = "15.3"
  instance_class  = "db.r6g.large"
  allocated_storage = 100

  timeouts {
```

```
    create = "60m"
    update = "80m"
    delete = "40m"
  }
}

resource "aws_eks_cluster" "main" {
  name = "prod-cluster"
  # ...
  timeouts {
    create = "30m"
    delete = "15m"
  }
}
```

#023 Error: Error creating/updating resource: EntityAlreadyExists

Root Cause Analysis (RCA)

Terraform is trying to create a resource that already exists in the cloud provider, but it isn't tracked in the Terraform state. This happens when resources are created manually or by another pipeline outside of Terraform.

Solution

Import the existing resource into the Terraform state using `terraform import`, then run `terraform plan` to verify there are no unintended changes. Going forward, manage all resources through Terraform.

Example

```
# Import an existing AWS S3 bucket
terraform import aws_s3_bucket.main my-existing-bucket-name

# Import an existing EC2 instance
terraform import aws_instance.web i-1234567890abcdef0

# Import an existing IAM role
terraform import aws_iam_role.admin existing-role-name

# After import, run plan to check for drift
terraform plan # Should show "No changes" if config matches
```

#024 Error: Error deleting resource: DependencyViolation

Root Cause Analysis (RCA)

A resource cannot be deleted because other resources depend on it outside of Terraform's knowledge. For example, an EC2 instance using a security group, a VPC with subnets still attached, or an S3 bucket with objects still present.

Solution

Delete dependent resources first, or configure the resource to automatically handle cleanup. For S3 buckets, enable `force_destroy = true`. For VPCs, ensure all subnets, gateways, and instances are removed first.

Example

```
# S3 bucket with force_destroy for clean deletion
resource "aws_s3_bucket" "data" {
  bucket      = "my-data-bucket"
  force_destroy = true # Deletes all objects on destroy
}
```



```
# Proper VPC teardown order using depends_on
resource "aws_internet_gateway" "main" {
  vpc_id = aws_vpc.main.id
}
resource "aws_subnet" "public" {
  vpc_id = aws_vpc.main.id
  # ...
}
# Terraform resolves order, but explicit depends_on helps
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
}
```

#025 Error: Unsupported Terraform Core version – required_version constraint does not allow the current version

Root Cause Analysis (RCA)

The `required_version` constraint in the `terraform` block excludes the currently installed Terraform CLI version. This prevents teams from accidentally applying changes with incompatible versions.

Solution

Upgrade or downgrade the Terraform CLI to a version that satisfies the constraint, or adjust the constraint if the team agrees on a different version. Use `tfenv` for easy version switching.

Example

```
# In versions.tf
terraform {
  required_version = ">= 1.5.0, < 2.0.0"

  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

# Switch version with tfenv
tfenv install 1.7.0
tfenv use 1.7.0
terraform version # Confirm: Terraform v1.7.0
```

Resource-Specific Errors

Errors #026–#030

#026 Error: Error creating S3 bucket: BucketAlreadyExists

Root Cause Analysis (RCA)

S3 bucket names are globally unique across all AWS accounts. The desired bucket name is already taken by another AWS account, or by a previous deployment that wasn't cleaned up in state.

Solution

Choose a unique bucket name by incorporating account ID, region, or a unique suffix. Use the AWS account ID in the bucket name to ensure uniqueness. Avoid generic names like 'my-app-bucket'.

Example

```
data "aws_caller_identity" "current" {}
data "aws_region" "current" {}

resource "aws_s3_bucket" "app_data" {
  # Include account ID and region for global uniqueness
  bucket = "myapp-data-${data.aws_caller_identity.current.account_id}-${data.aws_region.current.name}"

  tags = {
    Environment = var.environment
    ManagedBy   = "Terraform"
  }
}
```

#027 Error: Error creating Security Group Rule: InvalidPermission.Duplicate

Root Cause Analysis (RCA)

Terraform is attempting to create a security group rule that already exists on the security group. This can happen when mixing inline ingress/egress blocks in `aws_security_group` with standalone `aws_security_group_rule` resources.

Solution

Never mix inline security group rules with standalone `aws_security_group_rule` resources for the same group. Choose one approach and stick with it. Use `lifecycle ignore_changes` if drift is expected.

Example

```
# Choose ONE approach per security group

# Approach 1: Inline rules (simpler)
resource "aws_security_group" "web" {
  name = "web-sg"
  ingress {
    from_port = 443
    to_port   = 443
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

```
}

# Approach 2: Standalone rules (more flexible)
resource "aws_security_group" "api" {
  name = "api-sg"
  # No inline rules here!
}
resource "aws_security_group_rule" "https_in" {
  type      = "ingress"
  from_port = 443
  to_port   = 443
  protocol  = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
  security_group_id = aws_security_group.api.id
}
```

#028 Error: InvalidSubnetID.NotFound – The subnet ID does not exist

Root Cause Analysis (RCA)

The subnet ID referenced in a resource doesn't exist in the specified region or account. This often happens with hardcoded subnet IDs that differ between environments (dev vs prod), or when subnets are deleted outside Terraform.

Solution

Use data sources to look up subnet IDs dynamically instead of hardcoding them. Reference subnets created within the same Terraform configuration to ensure they always exist.

Example

```
# Dynamic lookup using data source
data "aws_subnets" "private" {
  filter {
    name = "vpc-id"
    values = [aws_vpc.main.id]
  }
  tags = {
    Type = "private"
  }
}

resource "aws_instance" "app" {
  ami      = "ami-0c55b159cbf1f0"
  instance_type = "t3.micro"
  subnet_id = data.aws_subnets.private.ids[0]
}
```

#029 Error: Error creating RDS instance: DBInstanceAlreadyExists

Root Cause Analysis (RCA)

An RDS instance with the same identifier already exists in the account/region. The identifier must be unique within a region. This often occurs when a destroy operation fails midway and the RDS instance remains.

Solution

Choose a unique identifier, import the existing instance if you want to manage it, or delete it manually via the AWS Console before re-running apply. Use random suffixes for test environments.

Example

```
resource "random_id" "db_suffix" {
  byte_length = 4
}

resource "aws_db_instance" "main" {
  identifier      = "myapp-db-${var.environment}-${random_id.db_suffix.hex}"
  engine         = "postgres"
  engine_version = "15"
  instance_class = "db.t3.medium"
  allocated_storage = 20
  db_name        = "myappdb"
  username       = "dbadmin"
  password       = var.db_password
  skip_final_snapshot = true
}
```

#030 Error: Error creating IAM Policy: MalformedPolicyDocument

Root Cause Analysis (RCA)

The IAM policy JSON document contains a syntax error, references an invalid ARN, uses incorrect action names, or violates IAM policy structure requirements. This is often a JSON formatting issue.

Solution

Validate the policy JSON using the AWS IAM Policy Simulator or `aws iam simulate-policy`. Use Terraform's `jsonencode()` to generate policy documents programmatically, or use the `aws_iam_policy_document` data source.

Example

```
# Use data source for type-safe policy generation
data "aws_iam_policy_document" "s3_read" {
  statement {
    sid = "AllowS3Read"
    effect = "Allow"
    actions = [
      "s3:GetObject",
      "s3:ListBucket"
    ]
    resources = [
      aws_s3_bucket.data.arn,
      "${aws_s3_bucket.data.arn}/*"
    ]
  }
}

resource "aws_iam_policy" "s3_read" {
  name = "s3-read-policy"
  policy = data.aws_iam_policy_document.s3_read.json
}
```

Modules

Errors #031–#035

#031 Error: Module source not found – no module source configuration

Root Cause Analysis (RCA)

The source argument is missing from a module block, or the specified source path doesn't exist. Local module paths are relative to the calling configuration file.

Solution

Add the source argument to every module block. For local modules, use a relative path starting with `./`. Run `terraform init` after adding or changing module sources.

Example

```
# Local module (relative path)
module "networking" {
  source = "./modules/networking"
  vpc_cidr = "10.0.0.0/16"
}

# Registry module
module "vpc" {
  source = "terraform-aws-modules/vpc/aws"
  version = "~> 5.0"
  name    = "prod-vpc"
  cidr    = "10.0.0.0/16"
}

# Git source
module "custom" {
  source = "git::https://github.com/org/tf-module.git?ref=v1.2.0"
}
```

#032 Error: Unsupported module output – module does not have an output 'X'

Root Cause Analysis (RCA)

The calling configuration references `module.name.output_name`, but the module's `outputs.tf` file doesn't declare an output with that exact name. Typos and version mismatches are the most common causes.

Solution

Verify the output name against the module's `outputs.tf` file. Check if you are using the correct module version that includes the expected output. Add the missing output to the module if you control it.

Example

```
# In module's outputs.tf
output "vpc_id" {
  description = "The ID of the VPC"
  value       = aws_vpc.main.id
}
```

```
output "private_subnet_ids" {
  description = "List of private subnet IDs"
  value      = aws_subnet.private[*].id
}

# In root module – must match exactly
module "network" {
  source = "./modules/network"
}

resource "aws_instance" "app" {
  subnet_id = module.network.private_subnet_ids[0]
  # NOT: module.network.private_subnets (wrong name)
}
```

#033 Error: Module input variable 'X' is not declared

Root Cause Analysis (RCA)

The module block passes a variable that the module's variables.tf doesn't declare. This can happen after module refactoring where variable names changed, or when using a newer module that renamed inputs.

Solution

Review the module's variables.tf to find the correct variable names. Remove undeclared variables from the module block or add the variable declaration to the module's variables.tf if you own it.

Example

```
# Module variables.tf
variable "instance_type" {
  description = "EC2 instance type"
  type       = string
  default    = "t3.micro"
}

variable "environment" {
  description = "Deployment environment"
  type       = string
}

# Root module calling – must match variable names
module "compute" {
  source      = "./modules/compute"
  instance_type = "t3.small" # matches variable name
  environment  = "production" # matches variable name
  # instance_size = "t3.small" # WRONG – not declared
}
```

#034 Error: Cycle – module graph contains a cycle

Root Cause Analysis (RCA)

Two or more modules reference each other's outputs, creating a circular dependency. Terraform cannot determine the execution order when cycles exist.

Solution

Refactor to break the cycle. Extract the shared dependency into a separate base module. Pass values down through variables instead of creating circular output references.

Example

```
# WRONG – Module A references Module B, Module B references Module A

# CORRECT – Extract shared resource to base module
module "base" {
  source = "./modules/base"
}

module "module_a" {
  source = "./modules/module_a"
  base_id = module.base.id
}

module "module_b" {
  source = "./modules/module_b"
  base_id = module.base.id
  # Not referencing module_a!
}
```

#035 Error: Module not installed – run 'terraform init' to install modules

Root Cause Analysis (RCA)

A module source has been added or changed in the configuration, but terraform init hasn't been run to download/install the new module. The .terraform/modules directory is missing or out of date.

Solution

Run terraform init to download all module sources. In CI/CD pipelines, always run terraform init before plan or apply. Use -upgrade flag to update to newer module versions.

Example

```
# Always run init after adding/changing modules
terraform init

# Update modules to latest compatible versions
terraform init -upgrade

# In CI/CD Makefile
.PHONY: init plan apply
init:
    terraform init -input=false
plan: init
    terraform plan -out=tfplan
apply:
    terraform apply -auto-approve tfplan
```

Workspace & Environment

Errors #036–#040

#036 Error: Workspace does not exist

Root Cause Analysis (RCA)

A terraform workspace select or terraform workspace delete command references a workspace that doesn't exist yet. The default workspace always exists, but custom workspaces must be created first.

Solution

Create the workspace with terraform workspace new <name> before selecting it. Use terraform workspace list to see all available workspaces.

Example

```
# List existing workspaces
terraform workspace list

# Create and switch to a new workspace
terraform workspace new staging

# Select an existing workspace
terraform workspace select production

# Show current workspace in config
locals {
  environment = terraform.workspace
  instance_type = terraform.workspace == "production" ? "t3.large" : "t3.micro"
}
```

#037 Error: terraform.tfvars file not found in module directory

Root Cause Analysis (RCA)

Terraform looks for terraform.tfvars (and *.auto.tfvars) only in the root module directory, not in child module directories. Placing a .tfvars file in a module directory has no effect.

Solution

Move variable definition files to the root module directory. Pass variables to child modules using the module block's input arguments. Use -var-file flag when calling from CI/CD.

Example

```
# Correct file structure
project/
├── main.tf
├── variables.tf
├── terraform.tfvars    # Root-level vars file
├── environments/
│   ├── prod.tfvars    # Environment-specific vars
│   └── staging.tfvars
└── modules/
```



```
└─ networking/      # NO tfvars files here
```

```
# Apply with environment-specific vars
terraform plan -var-file="environments/prod.tfvars"
```

#038 Error: No value for required variable – variable 'X' is not set

Root Cause Analysis (RCA)

A required variable (one without a default value) hasn't been assigned a value through tfvars file, -var flag, TF_VAR_ environment variable, or interactive prompt (when running without -input=false).

Solution

Provide the variable value via one of the accepted methods. In CI/CD, set TF_VAR_* environment variables or use -var-file. Add sensible defaults for non-sensitive optional variables.

Example

```
# Option 1: terraform.tfvars
region    = "us-east-1"
environment = "production"

# Option 2: Environment variable
export TF_VAR_region="us-east-1"
export TF_VAR_environment="production"

# Option 3: CLI flag
terraform plan \
  -var="region=us-east-1" \
  -var="environment=production"

# Option 4: Add default to variable
variable "region" {
  type    = string
  default = "us-east-1"
}
```

#039 Error: Invalid value for variable – Variable validation failed

Root Cause Analysis (RCA)

A variable's value doesn't pass the custom validation rules defined in its validation block. Terraform evaluates validations before applying and rejects non-conforming values.

Solution

Check the variable's validation block for the allowed values or pattern. Pass a value that satisfies all validation conditions. Improve error messages in the validation block to help users understand requirements.

Example

```
variable "environment" {
  type      = string
  description = "Deployment environment"

  validation {
    condition     = contains(["dev", "staging", "production"], var.environment)
    error_message = "Environment must be one of: dev, staging, production."
  }
}
```

```
variable "instance_type" {
  type = string
  validation {
    condition = can(regex("^t[23]\\.(micro|small|medium|large)$", var.instance_type))
    error_message = "Must be a t2 or t3 instance type (micro through large)."
  }
}
```

#040 Error: Sensitive variable value not allowed in output without sensitive = true

Root Cause Analysis (RCA)

An output value references a sensitive variable or resource attribute but doesn't mark itself as sensitive. Terraform prevents accidental exposure of sensitive data in plan output.

Solution

Mark outputs that contain sensitive data with `sensitive = true`. This prevents the value from appearing in plan/apply output while still allowing it to be used by other modules or operations.

Example

```
# Variable marked as sensitive
variable "db_password" {
  type = string
  sensitive = true
}

# Output must also be marked sensitive
output "db_connection_string" {
  sensitive = true
  value = "postgres://admin:${var.db_password}@${aws_db_instance.main.endpoint}/mydb"
}

# Sensitive resource attributes propagate automatically
output "rds_password" {
  sensitive = true
  value = aws_db_instance.main.password
}
```

Network & Permissions

Errors #041–#045

#041 Error: UnauthorizedOperation – You are not authorized to perform this operation

Root Cause Analysis (RCA)

The IAM credentials used by Terraform lack the necessary permissions to create, modify, or delete the target resource. The principal (user/role) is missing IAM policy statements for the attempted action.

Solution

Attach appropriate IAM policies to the user or role. Use IAM policy simulator to verify permissions. Follow least-privilege principle and grant only required permissions.

Example

```
# Minimum IAM policy for Terraform to manage EC2
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "EC2Management",
      "Effect": "Allow",
      "Action": [
        "ec2:Describe*",
        "ec2:RunInstances",
        "ec2:TerminateInstances",
        "ec2:CreateTags",
        "ec2:ModifyInstanceAttribute"
      ],
      "Resource": "*"
    }
  ]
}

# Use aws iam simulate-principal-policy to test
aws iam simulate-principal-policy \
  --policy-source-arn arn:aws:iam::123456789012:user/terraform \
  --action-names ec2:RunInstances
```

#042 Error: Error creating VPC: VpcLimitExceeded

Root Cause Analysis (RCA)

AWS has a default limit of 5 VPCs per region. When this limit is reached, new VPC creation fails. This is a soft limit that can be increased.

Solution

Delete unused VPCs, or request a service quota increase via AWS Service Quotas console. Use data sources to reference existing VPCs instead of creating new ones where possible.

Example

```
# Request quota increase via CLI
aws service-quotas request-service-quota-increase \
--service-code vpc \
--quota-code L-F678F1CE \
--desired-value 10 \
--region us-east-1

# Reuse existing VPC with data source
data "aws_vpc" "existing" {
  tags = {
    Name = "shared-vpc"
  }
}

resource "aws_subnet" "app" {
  vpc_id   = data.aws_vpc.existing.id
  cidr_block = "10.0.10.0/24"
}
```

#043 Error: Error creating EIP: AddressLimitExceeded – Too many addresses allocated

Root Cause Analysis (RCA)

The default limit for Elastic IP addresses is 5 per region. NAT gateways and instances requiring public IPs can exhaust this quickly in larger deployments.

Solution

Release unused EIPs, design architectures that share NAT gateways across subnets, or request a quota increase through AWS Service Quotas.

Example

```
# Release unassociated EIPs via CLI
aws ec2 describe-addresses \
--filters "Name=association-id,Values=" | \
jq -r '.Addresses[].AllocationId'

# Share a single NAT Gateway
resource "aws_eip" "nat" {
  domain = "vpc"
}

resource "aws_nat_gateway" "main" {
  allocation_id = aws_eip.nat.id
  subnet_id     = aws_subnet.public.id
}

# All private subnets share one NAT Gateway
resource "aws_route_table" "private" {
  count = length(var.private_subnet_cidrs)
  vpc_id = aws_vpc.main.id

  route {
    cidr_block     = "0.0.0.0/0"
    nat_gateway_id = aws_nat_gateway.main.id
  }
}
```

#044 Error: dial tcp: connection refused – unable to connect to provider API endpoint

Root Cause Analysis (RCA)

Network connectivity issues prevent Terraform from reaching the provider's API. This can be caused by incorrect proxy settings, VPN not connected, security group rules blocking outbound traffic, or DNS resolution failures.

Solution

Verify network connectivity to the API endpoint. Check proxy settings with HTTPS_PROXY. For execution in private environments, ensure NAT gateway or VPC endpoints are configured.

Example

```
# Verify connectivity
curl -v https://ec2.us-east-1.amazonaws.com

# Set proxy if needed
export HTTPS_PROXY="http://proxy.company.com:8080"
export NO_PROXY="localhost,127.0.0.1,.company.internal"

# Use VPC endpoints in private subnets (no internet needed)
resource "aws_vpc_endpoint" "s3" {
  vpc_id      = aws_vpc.main.id
  service_name = "com.amazonaws.us-east-1.s3"
  vpc_endpoint_type = "Gateway"
  route_table_ids = [aws_route_table.private.id]
}
```

#045 Error: ResourceInUse – The resource is currently in use and cannot be deleted

Root Cause Analysis (RCA)

A resource has active dependencies that prevent deletion. For example, an AMI with running instances, a KMS key with active aliases, a certificate with attached load balancers, or a security group in use by an EC2 instance.

Solution

Identify and remove all dependent resources first. Terraform should handle this automatically if dependencies are modeled correctly, but manual resources outside Terraform may need cleanup.

Example

```
# Proper teardown order with explicit depends_on
resource "aws_lb_listener" "https" {
  certificate_arn = aws_acm_certificate.main.arn
  load_balancer_arn = aws_lb.main.arn
  # ...
}

# Certificate can only be deleted after listener is gone
resource "aws_acm_certificate" "main" {
  domain_name = "example.com"
  # Terraform will delete listener before certificate
}

# For KMS keys, schedule deletion
resource "aws_kms_key" "main" {
  deletion_window_in_days = 7 # Min 7, Max 30
  enable_key_rotation     = true
}
```

Data Sources

Errors #046–#050

#046 Error: Your query returned no results – data source returned 0 results

Root Cause Analysis (RCA)

A data source filter returns an empty result set. Common causes include incorrect filter values, resources in different regions, resources not yet created, or typos in tag names/values.

Solution

Verify the filter values match existing resources. Check the region setting. Use AWS CLI to confirm the resource exists with the expected attributes before referencing it in a data source.

Example

```
# Verify resource exists first
aws ec2 describe-instances \
  --filters "Name=tag:Name,Values=my-app-server"

# Data source with specific filters
data "aws_instance" "app" {
  filter {
    name   = "tag:Name"
    values = ["my-app-server"]
  }
  filter {
    name   = "instance-state-name"
    values = ["running"]
  }
}

# Use most_recent for AMIs
data "aws_ami" "ubuntu" {
  most_recent = true
  owners     = ["099720109477"] # Canonical
  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu-*-22.04-amd64-server-*"]
  }
}
```

#047 Error: data source returned more than one result

Root Cause Analysis (RCA)

A data source expects to return a single resource but the filter matches multiple resources. This is common with AMI lookups, instance searches, or when querying resources without unique identifiers.

Solution

Add more specific filters to narrow the results to exactly one resource. For AMIs, use `most_recent = true` along with specific owner and name filters. For instances, filter by unique tags.

Example

```
# Too broad – may match multiple AMIs
data "aws_ami" "bad" {
  filter {
    name = "name"
    values = ["ubuntu-*"] # Too broad!
  }
}

# Correct – narrow down with multiple filters
data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"]
  filter {
    name = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-*"]
  }
  filter {
    name = "virtualization-type"
    values = ["hvm"]
  }
  filter {
    name = "root-device-type"
    values = ["ebs"]
  }
}
```

#048 Error: Error reading data source: 404 Not Found**Root Cause Analysis (RCA)**

The data source is querying a resource (like an SSM parameter, Secrets Manager secret, or remote state) that doesn't exist. The resource may have been deleted, or the path/name is incorrect.

Solution

Verify the resource exists at the specified path. Check for typos in paths, ensure you're querying the right region, and verify the credentials have read permissions.

Example

```
# Verify SSM parameter exists
aws ssm get-parameter --name "/myapp/prod/db_password"

# Data source for SSM parameter
data "aws_ssm_parameter" "db_password" {
  name      = "/myapp/${var.environment}/db_password"
  with_decryption = true
}

# Data source for remote state
data "terraform_remote_state" "network" {
  backend = "s3"
  config = {
    bucket = "my-tf-state"
    key    = "network/terraform.tfstate"
    region = "us-east-1"
  }
}

resource "aws_instance" "app" {
```

```
subnet_id = data.terraform_remote_state.network.outputs.private_subnet_id
}
```

#049 Error: Self-referential block – a resource cannot reference its own attributes during planning

Root Cause Analysis (RCA)

A resource block references its own computed attributes (like id or arn) in an argument that needs to be known before creation. For example, referencing `aws_s3_bucket.main.arn` within the same resource's policy.

Solution

Break the self-reference by creating a separate resource for the dependent configuration. For bucket policies, use `aws_s3_bucket_policy` as a separate resource that references the bucket's ARN.

Example

```
# Wrong – bucket referencing its own ARN
resource "aws_s3_bucket" "bad" {
  bucket = "my-bucket"
  policy = jsonencode({ # Can't reference self!
    Statement = [{ Resource = "${aws_s3_bucket.bad.arn}/*" }]
  })
}

# Correct – separate policy resource
resource "aws_s3_bucket" "good" {
  bucket = "my-bucket"
}

resource "aws_s3_bucket_policy" "good" {
  bucket = aws_s3_bucket.good.id # OK to reference
  policy = jsonencode({
    Statement = [{
      Resource = "${aws_s3_bucket.good.arn}/*"
    }]
  })
}
```

#050 Error: Cannot use depends_on with data sources that use expressions referring to other resources

Root Cause Analysis (RCA)

A data source has both implicit references to resources (through expressions) and an explicit `depends_on`, which can create ambiguous dependency ordering that Terraform cannot resolve safely.

Solution

Remove the explicit `depends_on` if implicit references already establish the dependency, or restructure to use one dependency mechanism. For strict ordering, prefer the implicit reference approach.

Example

```
# Implicit reference is sufficient
data "aws_ami" "latest" {
  most_recent = true
  owners     = ["self"]
  filter {
    name = "tag:Project"
    values = [aws_codebuild_project.build.name] # Implicit dep
  }
}
```



```
# No depends_on needed – implicit ref creates dependency
}

# When you need explicit ordering without reference
data "aws_instances" "workers" {
  filter {
    name   = "tag:Role"
    values = ["worker"]
  }
  depends_on = [aws_autoscaling_group.workers]
}
```

Terraform Cloud & Enterprise

Errors #051–#053

#051 Error: Required token could not be found – no credentials found for app.terraform.io

Root Cause Analysis (RCA)

Terraform Cloud operations require a user or team API token that hasn't been configured. The token can be provided via credentials block in `.terraformrc` or the `TFE_TOKEN` environment variable.

Solution

Log in using `terraform login` to store credentials, or set `TF_TOKEN_app_terraform_io` environment variable. For CI/CD, use `TFE_TOKEN` environment variable with a team or organization token.

Example

```
# Interactive login
terraform login

# Environment variable (for CI/CD)
export TFE_TOKEN_app_terraform_io="<your-token>"

# Or configure .terraformrc
credentials "app.terraform.io" {
  token = "<your-token>"
}

# Terraform Cloud backend
terraform {
  cloud {
    organization = "my-org"
    workspaces {
      name = "prod-infra"
    }
  }
}
```

#052 Error: Workspace is locked – another run is currently in progress

Root Cause Analysis (RCA)

Terraform Cloud/Enterprise serializes runs per workspace. A run (plan, apply, or destroy) is currently in progress, so new runs are queued or blocked.

Solution

Wait for the current run to complete in the Terraform Cloud UI. If the run is stuck, cancel it through the UI or API. Implement run triggers carefully to avoid overlapping runs.

Example

```
# Cancel a stuck run via TFC API
curl -s \
  --header "Authorization: Bearer $TFE_TOKEN" \
```

```
--header "Content-Type: application/vnd.api+json" \  
--request POST \  
"https://app.terraform.io/api/v2/runs/$RUN_ID/actions/cancel"  
  
# Set parallelism in workspace settings  
# Or via CLI flag  
terraform apply -parallelism=5
```

#053 Error: Missing or invalid credentials for private registry module

Root Cause Analysis (RCA)

A module source references a private Terraform Registry but the necessary authentication credentials haven't been configured. This occurs with Terraform Cloud private module registries.

Solution

Configure authentication for the private registry in the .terraformrc file or use TF_TOKEN_* environment variables. Ensure the token has access to the private module registry.

Example

```
# Configure private registry credentials  
credentials "my-registry.example.com" {  
  token = "my-access-token"  
}  
  
# Reference private registry module  
module "vpc" {  
  source = "my-registry.example.com/myorg/vpc/aws"  
  version = "~> 2.0"  
}  
  
# List available modules in private registry  
curl -H "Authorization: Bearer $TOKEN" \  
"https://my-registry.example.com/v1/modules?namespace=myorg"
```

Kubernetes Provider

Errors #054–#055

#054 Error: Post 'https://kubernetes.default.svc/api/v1/namespaces': connection refused

Root Cause Analysis (RCA)

The Kubernetes provider cannot connect to the cluster API server. The kubeconfig file is missing, pointing to the wrong cluster, or the cluster API endpoint is unreachable from the Terraform execution environment.

Solution

Ensure the kubeconfig is correctly configured and the cluster is accessible. When using EKS, generate the kubeconfig using `aws eks update-kubeconfig`. Set the `KUBECONFIG` environment variable if using a non-default path.

Example

```
# Update kubeconfig for EKS
aws eks update-kubeconfig \
  --region us-east-1 \
  --name my-eks-cluster

# Kubernetes provider with explicit config
provider "kubernetes" {
  host           = data.aws_eks_cluster.main.endpoint
  cluster_ca_certificate = base64decode(
    data.aws_eks_cluster.main.certificate_authority[0].data
  )
  token = data.aws_eks_cluster_auth.main.token
}

data "aws_eks_cluster" "main" {
  name = "my-eks-cluster"
}

data "aws_eks_cluster_auth" "main" {
  name = "my-eks-cluster"
}
```

#055 Error: namespaces 'X' not found – namespace does not exist

Root Cause Analysis (RCA)

A Kubernetes resource references a namespace that doesn't exist in the cluster. Namespaces must be created before resources that use them, and Terraform must manage the dependency order.

Solution

Create the namespace resource first or use `depends_on` to ensure proper ordering. Reference the namespace resource directly to establish an implicit dependency.

Example

```
resource "kubernetes_namespace" "app" {
  metadata {
```

```
name = "my-application"
labels = {
  environment = var.environment
}
}
}

resource "kubernetes_deployment" "app" {
  metadata {
    name      = "myapp"
    namespace = kubernetes_namespace.app.metadata[0].name # Implicit dep
  }
  spec {
    replicas = 3
    # ...
  }
}
```

Azure Provider

Errors #056–#057

#056 Error: A resource with the ID X already exists – to be managed via Terraform import

Root Cause Analysis (RCA)

Terraform is trying to create an Azure resource that already exists. The resource was created outside Terraform or by a previous run whose state was lost.

Solution

Import the existing resource into Terraform state using `terraform import` with the correct Azure resource ID. After importing, run `plan` to check for any configuration drift.

Example

```
# Get existing resource ID from Azure
az resource show \
  --resource-group myRG \
  --resource-type "Microsoft.Network/virtualNetworks" \
  --name myVNet \
  --query id -o tsv

# Import into Terraform state
terraform import azurerm_virtual_network.main \
  /subscriptions/${SUBSCRIPTION_ID}/resourceGroups/myRG/providers/Microsoft.Network/virtualNetworks/myVNet

# Verify
terraform plan # Should show no changes
```

#057 Error: Error creating Resource Group – 'QuotaExceeded' for region

Root Cause Analysis (RCA)

Azure subscription resource quotas have been reached for the target region. Each subscription has limits on vCPUs, storage accounts, virtual networks, and other resources per region.

Solution

Request a quota increase through the Azure Portal under Subscriptions > Usage + Quotas. Alternatively, distribute resources across multiple regions or clean up unused resources.

Example

```
# Check current quota usage
az vm list-usage \
  --location eastus \
  --query "[?contains(name.localizedValue,'vCPUs')].{Name:name.localizedValue, Used:currentValue, Limit:limit}" \
  --output table

# Azure resource group with tags
resource "azurerm_resource_group" "main" {
  name     = "rg-myapp-prod"
  location = var.location
}
```

```
tags = {  
  Environment = "Production"  
  ManagedBy   = "Terraform"  
}
```

GCP Provider

Errors #058–#060

#058 Error: Error creating Project Service: googleapi: Error 403 – required 'serviceusage.services.enable' permission

Root Cause Analysis (RCA)

The GCP service account used by Terraform doesn't have permission to enable API services. Enabling APIs requires the Service Usage Admin role or similar permissions.

Solution

Grant the Service Usage Admin (roles/serviceusage.serviceUsageAdmin) role to the Terraform service account, or pre-enable required APIs and remove the google_project_service resources from Terraform.

Example

```
# Grant required roles to Terraform SA
gcloud projects add-iam-policy-binding PROJECT_ID \
  --member="serviceAccount:terraform@PROJECT_ID.iam.gserviceaccount.com" \
  --role="roles/serviceusage.serviceUsageAdmin"

# Enable APIs in Terraform
resource "google_project_service" "compute" {
  project = var.project_id
  service = "compute.googleapis.com"

  disable_on_destroy = false # Keep API enabled when resource is destroyed
}

resource "google_compute_instance" "main" {
  depends_on = [google_project_service.compute]
  # ...
}
```

#059 Error: Error creating Compute Instance: googleapi: Error 400 – Invalid value for field 'zone'

Root Cause Analysis (RCA)

The specified GCP zone doesn't exist or isn't available for the project. Zone names must match GCP's exact naming convention (e.g., us-central1-a, not us-central1-zone-a).

Solution

Use the correct zone format: region-letter (e.g., us-central1-a). List available zones using gcloud compute zones list. Use data sources to query available zones dynamically.

Example

```
# List available zones
gcloud compute zones list --filter="region:us-central1"

# Use data source for available zones
data "google_compute_zones" "available" {
```



```
project = var.project_id
region  = var.region
status  = "UP"
}

resource "google_compute_instance" "main" {
  name         = "my-instance"
  machine_type = "e2-medium"
  zone         = data.google_compute_zones.available.names[0]

  boot_disk {
    initialize_params {
      image = "debian-cloud/debian-11"
    }
  }
}
```

#060 Error: Error reading Service Account: 404 Not Found

Root Cause Analysis (RCA)

A GCP resource references a service account that doesn't exist yet. The service account creation may have failed previously, or the email format is incorrect.

Solution

Create the service account resource explicitly and reference it in dependent resources. Verify the service account email format: name@project.iam.gserviceaccount.com.

Example

```
resource "google_service_account" "app" {
  account_id = "myapp-sa"
  display_name = "MyApp Service Account"
  project    = var.project_id
}

resource "google_compute_instance" "app" {
  name         = "app-server"
  machine_type = "e2-medium"
  zone         = "us-central1-a"

  service_account {
    email = google_service_account.app.email # Implicit dependency
    scopes = ["cloud-platform"]
  }
}
```

Lifecycle & Meta-Arguments

Errors #061–#065

#061 Error: Error updating resource – changes to X forces replacement of resource

Root Cause Analysis (RCA)

An attribute that cannot be updated in-place has been modified. Terraform must destroy and recreate the resource. If the resource is critical (like a database), this unexpected replacement can cause downtime.

Solution

Use `lifecycle { create_before_destroy = true }` to minimize downtime. For critical resources, use `prevent_destroy = true` to block accidental replacements. Consider blue-green deployment strategies.

Example

```
resource "aws_instance" "web" {
  ami      = data.aws_ami.ubuntu.id
  instance_type = var.instance_type

  lifecycle {
    create_before_destroy = true # New instance before destroying old
    prevent_destroy       = false
    ignore_changes        = [ami] # Don't replace on AMI updates
  }
}

# For stateful resources, prevent accidental deletion
resource "aws_db_instance" "main" {
  identifier = "prod-db"
  # ...
  lifecycle {
    prevent_destroy = true # Requires -target to override
  }
}
```

#062 Error: Instance cannot be destroyed – lifecycle prevent_destroy is set

Root Cause Analysis (RCA)

A resource has `lifecycle { prevent_destroy = true }` configured, and terraform destroy or an apply that would destroy the resource was attempted. This is an intentional protection mechanism.

Solution

If destruction is intentional, temporarily remove the `prevent_destroy` setting and run `apply` first to update the state, then run `destroy`. Never remove `prevent_destroy` as a permanent solution for automated pipelines.

Example

```
# Temporarily remove to allow destruction
resource "aws_db_instance" "main" {
  identifier = "prod-db"
  # ...
}
```

```
lifecycle {
  prevent_destroy = false # Temporarily set to false
  # prevent_destroy = true # Re-enable after destruction
}
}
terraform apply # Updates lifecycle config in state
terraform destroy -target=aws_db_instance.main
```

#063 Error: ignore_changes applies to argument that does not exist

Root Cause Analysis (RCA)

The ignore_changes list in the lifecycle block references an attribute name that doesn't exist in the resource's schema. This can happen after provider upgrades that rename attributes.

Solution

Check the provider documentation for current attribute names. Remove or update stale attribute names from the ignore_changes list. Use all with caution as it ignores all changes.

Example

```
resource "aws_instance" "web" {
  ami      = data.aws_ami.ubuntu.id
  instance_type = "t3.micro"

  lifecycle {
    # Only use valid attribute names
    ignore_changes = [
      ami,          # Valid – ignore AMI updates
      tags["LastUpdated"], # Specific tag
    ]
    # DON'T use: ignore_changes = [ami_id] # Not a valid attribute
  }
}
```

#064 Error: count and for_each cannot both be set

Root Cause Analysis (RCA)

A resource or module block has both count and for_each meta-arguments set simultaneously. Terraform only allows one iteration mechanism per resource block.

Solution

Choose either count (for simple numerical repetition) or for_each (for map/set-based iteration) for a given resource. Convert to for_each when you need meaningful keys or want to avoid index-based references.

Example

```
# Wrong – both count and for_each
resource "aws_iam_user" "bad" {
  count = 3      # WRONG
  for_each = toset(var.users) # WRONG - can't have both

# Correct – use for_each for named resources
resource "aws_iam_user" "good" {
  for_each = toset(["alice", "bob", "charlie"])
  name    = each.key
}
```

```
# Or use count for simple numerical repetition
resource "aws_instance" "web" {
  count      = 3
  ami       = data.aws_ami.ubuntu.id
  instance_type = "t3.micro"
  tags = {
    Name = "web-${count.index}"
  }
}
```

#065 Error: Resource instance key required – because resource has count/for_each, an index is required

Root Cause Analysis (RCA)

A resource with count or for_each is referenced without specifying an index. You must use [count.index], [each.key], or a specific index like [0] or ["alice"] when referencing such resources.

Solution

Add the appropriate index when referencing resources with count or for_each. Use [*] to get all values as a list (splat expression) or a specific index to get one instance.

Example

```
resource "aws_instance" "web" {
  count      = 3
  ami       = "ami-xxx"
  instance_type = "t3.micro"
}

# Wrong – missing index
output "bad_ip" { value = aws_instance.web.private_ip }

# Correct options
output "first_ip" { value = aws_instance.web[0].private_ip }
output "all_ips"  { value = aws_instance.web[*].private_ip }
output "all_ids"  { value = [for i in aws_instance.web : i.id] }
```

Import & Migration

Errors #066–#070

#066 Error: Cannot import non-existent remote object

Root Cause Analysis (RCA)

terraform import was called with an ID that doesn't exist in the cloud provider. The resource may have been deleted, or the ID format is incorrect.

Solution

Verify the resource ID format (they vary by resource type) and confirm the resource exists using the cloud provider CLI or console. Check the provider documentation for the correct import ID format.

Example

```
# Always verify resource exists before importing
# EC2 instance ID format: i-xxxxxxxxxxxxxxxx
aws ec2 describe-instances --instance-ids i-1234567890abcdef0

# S3 bucket import – use bucket name only
terraform import aws_s3_bucket.main my-bucket-name

# RDS – use DB identifier
terraform import aws_db_instance.main mydb-identifier

# ALB – use ARN
terraform import aws_lb.main \
  arn:aws:elasticloadbalancing:us-east-1:123456789012:loadbalancer/app/my-alb/1234567890123456
```

#067 Error: Resource already managed by Terraform

Root Cause Analysis (RCA)

An import is being attempted for a resource that's already tracked in the Terraform state. Importing again would create a duplicate state entry.

Solution

Check the state with terraform state list before importing. If the resource already exists in state, run terraform plan to see if there's drift between state and reality, and use terraform apply to reconcile.

Example

```
# Check if resource is already in state
terraform state list | grep "aws_instance"

# Show resource details in state
terraform state show aws_instance.web

# If there's configuration drift, just run plan+apply
terraform plan # Shows drift
terraform apply # Reconciles
# Don't re-import!
```

#068 Error: terraform state mv – resource does not exist in state**Root Cause Analysis (RCA)**

The source address in terraform state mv doesn't match any resource in the state file. This can happen due to typos, incorrect index values with count/for_each, or the resource having been moved already.

Solution

Run terraform state list to see exact resource addresses. Copy the address verbatim including indices like [0] or ["key"] when using count or for_each.

Example

```
# List resources to get exact addresses
terraform state list

# Typical output:
# aws_instance.web[0]
# aws_instance.web[1]
# module.network.aws_vpc.main

# Move with exact address
terraform state mv \
  'aws_instance.web[0]' \
  'module.compute.aws_instance.web[0]'

# Rename a resource
terraform state mv \
  'aws_s3_bucket.old_name' \
  'aws_s3_bucket.new_name'
```

#069 Error: moved block source address X does not exist in configuration**Root Cause Analysis (RCA)**

A moved block in the configuration references a source address that no longer exists. This can happen after the source resource has been fully removed from both configuration and state.

Solution

Remove moved blocks once the migration is complete and all state files have been updated. Keep moved blocks only while the old addresses may still exist in state files that haven't been migrated.

Example

```
# In main.tf – use moved blocks for refactoring
moved {
  from = aws_instance.web
  to   = module.compute.aws_instance.web
}

# After running apply and verifying:
# 1. Confirm old address is gone: terraform state list
# 2. Remove the moved block from config
# 3. Commit the change

# Multiple moves can be combined
moved {
  from = aws_s3_bucket.legacy
```

```
to = aws_s3_bucket.app_data
}
```

#070 Error: Error running command – The terraform binary path was not found

Root Cause Analysis (RCA)

An automation script, wrapper, or CI/CD job cannot find the terraform binary. The binary isn't installed, isn't on the PATH, or the wrong directory is being searched.

Solution

Ensure Terraform is installed and accessible on PATH. Use absolute paths in CI/CD scripts, or use a tool like tfenv to manage installations. Verify with `which terraform`.

Example

```
# Verify installation
which terraform
terraform version

# Install on Ubuntu/Debian
wget -O- https://apt.releases.hashicorp.com/gpg | gpg --dearmor | \
  sudo tee /usr/share/keyrings/hashicorp-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] \
  https://apt.releases.hashicorp.com $(lsb_release -cs) main" | \
  sudo tee /etc/apt/sources.list.d/hashicorp.list
sudo apt update && sudo apt install terraform

# Add to PATH
export PATH="$PATH:/usr/local/bin"
echo 'export PATH="$PATH:/usr/local/bin"' >> ~/.bashrc
```

Outputs & Expressions

Errors #071–#075

#071 Error: Output refers to sensitive values – output 'X' refers to sensitive values

Root Cause Analysis (RCA)

An output value directly or indirectly references a sensitive input variable or a resource attribute marked as sensitive by the provider (e.g., passwords, secret keys) without marking the output as sensitive.

Solution

Mark the output as sensitive = true to prevent it from appearing in CLI output. If the value genuinely needs to be non-sensitive (e.g., for debugging), reassess whether sensitive inputs should flow into this output.

Example

```
variable "api_key" {
  type    = string
  sensitive = true
}

# Without sensitive = true, Terraform errors
output "api_endpoint" {
  sensitive = true
  value     = "https://api.example.com?key=${var.api_key}"
}

# Access sensitive output programmatically
terraform output -json api_endpoint | jq -r '.value'
```

#072 Error: The expression result is not known – a value is not available until apply

Root Cause Analysis (RCA)

An expression uses a value that's not known until apply time (like a computed resource ID) in a context where the value must be known during plan, such as count, for_each, or module source.

Solution

Avoid using unknown values in count and for_each. Use static values or data sources that resolve during plan. Restructure to separate the known-at-plan-time decisions from apply-time decisions.

Example

```
# Wrong – count depends on unknown value
resource "aws_subnet" "app" {
  count = aws_vpc.main.id == "" ? 0 : 3 # ID unknown at plan
}

# Correct – count based on known value
variable "subnet_count" { default = 3 }

resource "aws_subnet" "app" {
  count = var.subnet_count # Known at plan time
```



```
vpc_id      = aws_vpc.main.id # Unknown is OK here (not in count)
cidr_block  = cidrsubnet("10.0.0.0/16", 8, count.index)
availability_zone = data.aws_availability_zones.available.names[count.index]
}
```

#073 Error: Inconsistent conditional result types – values must have consistent types

Root Cause Analysis (RCA)

A conditional expression (condition ? true_val : false_val) returns different types in its true and false branches, e.g., a string in one branch and null or a list in the other.

Solution

Ensure both branches of a conditional return the same type. Use explicit type conversions like tostring(), tolist(), or tomap() to align types. Use null for the 'disabled' case but make sure the consuming attribute accepts null.

Example

```
# Wrong – string vs null type mismatch
variable "use_custom_sg" { type = bool }
variable "sg_id"         { type = string; default = "" }

resource "aws_instance" "web" {
  vpc_security_group_ids = var.use_custom_sg ? [var.sg_id] : null # OK for lists

  # Wrong type mismatch:
  # subnet_id = var.use_custom ? var.subnet_id : 123 # string vs number
}

# Correct – consistent types
locals {
  subnet_id = var.use_custom ? var.subnet_id : var.default_subnet_id
}
```

#074 Error: Invalid index – index out of range with X element(s)

Root Cause Analysis (RCA)

A list or tuple is accessed with an index that is greater than or equal to the list length. This happens when the list size changes between environments or when data sources return fewer results than expected.

Solution

Add length checks before indexing. Use try() to handle index errors gracefully. Design configurations to be resilient to varying list sizes, especially for availability zones and subnet IDs.

Example

```
# Unsafe – may fail if fewer than 3 AZs available
resource "aws_subnet" "az" {
  count      = 3
  availability_zone = data.aws_availability_zones.available.names[count.index]
}

# Safe – use available count
data "aws_availability_zones" "available" {
  state = "available"
}

locals {
```

```
az_count = min(3, length(data.aws_availability_zones.available.names))
}

resource "aws_subnet" "az" {
  count          = local.az_count
  availability_zone = data.aws_availability_zones.available.names[count.index]
  # ...
}
```

#075 Error: Cannot expand argument – argument 'X' is not permitted to use dynamic blocks here

Root Cause Analysis (RCA)

A dynamic block is used inside a context where dynamic blocks are not supported by the provider schema. Some nested blocks don't support dynamic expansion.

Solution

Check the provider documentation to confirm if the block supports dynamic expansion. If not, use static blocks or restructure the configuration to achieve the desired outcome through a different mechanism.

Example

```
# Dynamic blocks work for most repeating blocks
resource "aws_security_group" "web" {
  name = "web-sg"

  dynamic "ingress" {
    for_each = var.allowed_ports
    content {
      from_port = ingress.value
      to_port   = ingress.value
      protocol  = "tcp"
      cidr_blocks = ["0.0.0.0/0"]
    }
  }
}

variable "allowed_ports" {
  type = list(number)
  default = [80, 443, 8080]
}
```

Secrets & Security

Errors #076–#080

#076 Error: Credentials leaked in state file – sensitive data stored in plain text

Root Cause Analysis (RCA)

Sensitive values (passwords, API keys, certificates) are stored in the Terraform state file in plain text. The state file acts as a database of all resource attributes, including secrets returned by providers.

Solution

Enable encryption for state backend (S3 with SSE, Terraform Cloud, etc.). Use AWS Secrets Manager, HashiCorp Vault, or SSM Parameter Store to store secrets externally and reference them via data sources.

Example

```
# Store secrets in AWS Secrets Manager
resource "aws_secretsmanager_secret" "db_password" {
  name = "myapp/prod/db-password"
}

# Reference in resource (not stored in state)
data "aws_secretsmanager_secret_version" "db_password" {
  secret_id = aws_secretsmanager_secret.db_password.id
}

resource "aws_db_instance" "main" {
  password = data.aws_secretsmanager_secret_version.db_password.secret_string
}

# Always encrypt state at rest
terraform {
  backend "s3" {
    bucket = "my-state-bucket"
    encrypt = true # Enable SSE
    kms_key_id = "alias/terraform-state"
  }
}
```

#077 Error: Invalid KMS key – The specified KMS key does not exist

Root Cause Analysis (RCA)

A resource references a KMS key by ID or alias that doesn't exist in the account/region, has been deleted, or the executing role lacks kms:DescribeKey permission.

Solution

Verify the KMS key exists and the ARN/alias is correct. Ensure the Terraform execution role has kms:DescribeKey permission. Create the KMS key as a Terraform resource with proper key policies.

Example

```

resource "aws_kms_key" "main" {
  description      = "Main encryption key"
  deletion_window_in_days = 10
  enable_key_rotation = true

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Sid      = "Enable IAM User Permissions"
        Effect   = "Allow"
        Principal = { AWS = "arn:aws:iam::${data.aws_caller_identity.current.account_id}:root" }
        Action   = "kms:*"
        Resource = "*"
      }
    ]
  })
}

resource "aws_kms_alias" "main" {
  name          = "alias/myapp-main"
  target_key_id = aws_kms_key.main.key_id
}

resource "aws_s3_bucket_server_side_encryption_configuration" "main" {
  bucket = aws_s3_bucket.data.id
  rule {
    apply_server_side_encryption_by_default {
      kms_master_key_id = aws_kms_key.main.arn
      sse_algorithm     = "aws:kms"
    }
  }
}

```

#078 Error: Error creating Secret – SecretExistsException: secret already exists

Root Cause Analysis (RCA)

AWS Secrets Manager retains deleted secrets for a recovery window (7-30 days by default). Creating a new secret with the same name fails during this window.

Solution

Use a different secret name, recover the existing secret, or force-delete the secret using `--force-delete-without-recovery` flag. Append environment or version identifiers to secret names.

Example

```

# Force delete a secret (bypasses recovery window)
aws secretsmanager delete-secret \
  --secret-id myapp/prod/api-key \
  --force-delete-without-recovery

# Or recover the secret
aws secretsmanager restore-secret \
  --secret-id myapp/prod/api-key

# Use versioned names to avoid conflicts
resource "aws_secretsmanager_secret" "api_key" {
  name          = "myapp/${var.environment}/api-key-v2"
  recovery_window_in_days = 0 # Force immediate deletion on destroy
}

```

#079 Error: Invalid CIDR notation – 'X' is not a valid CIDR block**Root Cause Analysis (RCA)**

A CIDR block value is malformed. Common mistakes include host bits set in the network address (e.g., 10.0.1.0/8 instead of 10.0.0.0/8), missing prefix length, or using IP ranges instead of CIDR notation.

Solution

Use valid CIDR notation where host bits are zero. The cidrsubnet() function can help generate valid subnets. Validate with regex in variable validation blocks.

Example

```
# Wrong CIDR values
# 10.0.1.0/8 – host bits set (should be 10.0.0.0/8)
# 10.0.0.1-10.0.0.10 – range notation not valid
# 10.0.0.0 – missing prefix length

# Correct
variable "vpc_cidr" {
  type = string
  default = "10.0.0.0/16"

  validation {
    condition = can(cidrhost(var.vpc_cidr, 0))
    error_message = "Must be a valid IPv4 CIDR block."
  }
}

# Generate subnets programmatically
locals {
  private_subnets = [
    cidrsubnet(var.vpc_cidr, 8, 0), # 10.0.0.0/24
    cidrsubnet(var.vpc_cidr, 8, 1), # 10.0.1.0/24
    cidrsubnet(var.vpc_cidr, 8, 2), # 10.0.2.0/24
  ]
}
```

#080 Error: CIDR block conflict – existing CIDR range overlaps with the new CIDR**Root Cause Analysis (RCA)**

A new subnet or VPC CIDR range overlaps with an existing one. AWS and most cloud providers require non-overlapping CIDR ranges for subnets within a VPC and for VPC peering.

Solution

Plan your IP address space carefully. Use a consistent subnetting scheme. Tools like IPAM (IP Address Management) services help track and assign non-overlapping CIDRs across environments.

Example

```
# Systematic CIDR allocation
variable "vpc_cidr" { default = "10.0.0.0/16" }

locals {
  # /24 subnets: .0, .1, .2 for private; .100, .101, .102 for public
  private_cidrs = [for i in range(3) : cidrsubnet(var.vpc_cidr, 8, i)]
  public_cidrs = [for i in range(3) : cidrsubnet(var.vpc_cidr, 8, i + 100)]
}
```

```
# Output for verification
output "subnet_plan" {
  value = {
    private = local.private_cidrs
    public  = local.public_cidrs
  }
}
```

CIDR & Networking Logic

Errors #079–#080

Terraform Functions

Errors #081–#085

#081 Error: Error in function call – argument to 'lookup' is not a map

Root Cause Analysis (RCA)

The `lookup()` function's first argument is not a map type. It's being called with a list, string, or other non-map value, which is a type mismatch.

Solution

Ensure the first argument to `lookup()` is a map. If using a list, use `element()` or list indexing instead. Use `tomap()` to convert compatible objects if needed.

Example

```
# Wrong – lookup on a list
variable "instances" { default = ["t3.micro", "t3.small"] }
locals { bad = lookup(var.instances, "key", "default") }

# Correct – lookup on a map
variable "instance_types" {
  default = {
    dev = "t3.micro"
    prod = "t3.large"
  }
}

locals {
  instance_type = lookup(var.instance_types, var.environment, "t3.micro")
}

# Modern alternative using try with map access
locals {
  safe_type = try(var.instance_types[var.environment], "t3.micro")
}
```

#082 Error: Error in function call – argument to 'jsonencode' includes an attribute with a null value

Root Cause Analysis (RCA)

`jsonencode()` is called with an object that contains null values in attributes. While Terraform supports null, JSON encoding of null may produce unexpected results in policy documents or API calls.

Solution

Filter out null values before encoding, or provide default values. Use `compact()` for lists and conditionally include map keys using dynamic construction.

Example

```
# Remove null values from a map before encoding
locals {
  config = {
```



```

name = var.name
region = var.region
# Optional field – only include if set
kms_key = var.kms_key_id != null ? var.kms_key_id : null
}
# Filter nulls
clean_config = { for k, v in local.config : k => v if v != null }
}

output "encoded" {
  value = jsonencode(local.clean_config)
}

```

#083 Error: Error in function call – argument to 'file' must be a string

Root Cause Analysis (RCA)

The file() function receives a non-string argument, or references a file path that doesn't exist at the time Terraform evaluates the configuration. Relative paths are resolved from the module directory.

Solution

Ensure the file path is a string literal or a variable resolving to a string. Use path.module to build relative paths correctly. Verify the file exists before running Terraform.

Example

```

# Path relative to module directory
resource "aws_key_pair" "deployer" {
  key_name = "deployer-key"
  public_key = file("${path.module}/keys/deployer.pub")
}

# Read file content into a local
locals {
  user_data_script = file("${path.module}/scripts/setup.sh")
  env_config = filebase64("${path.module}/config/app.conf")
}

# Check if file exists (requires Terraform 1.4+)
locals {
  config_exists = fileexists("${path.module}/optional-config.json")
}

```

#084 Error: Error in function call – argument to 'base64decode' is not a valid base64 string

Root Cause Analysis (RCA)

The base64decode() function receives a string that isn't valid base64 encoding. This can happen when the value contains newlines, spaces, or is the raw value rather than its base64 encoding.

Solution

Ensure the input string is properly base64 encoded. Use base64encode() to encode values before passing to resources that expect base64. Strip whitespace from multi-line base64 strings.

Example

```

# Encode a file to base64
resource "aws_instance" "web" {
  ami = "ami-xxx"
}

```

```
instance_type = "t3.micro"

# file() returns string; filebase64() returns base64-encoded string
user_data = filebase64("${path.module}/scripts/init.sh")
# Equivalent: user_data = base64encode(file("scripts/init.sh"))
}

# Decode base64 from a data source
data "aws_secretsmanager_secret_version" "cert" {
  secret_id = "my-certificate"
}

locals {
  # Secrets can be stored as base64
  decoded_cert = base64decode(data.aws_secretsmanager_secret_version.cert.secret_string)
}
```

#085 Error: Error in function call – argument to 'tomap' cannot include values of multiple types

Root Cause Analysis (RCA)

The `tomap()` function requires all values in the collection to be of the same type. Mixing strings and numbers, or strings and booleans, causes a type error because Terraform maps are homogeneous.

Solution

Ensure all values in a map are the same type. Convert values to strings using `tostring()` if you need a mixed-type map, or use an object type if you need heterogeneous attributes.

Example

```
# Wrong – mixed types in map
locals {
  bad_map = tomap({
    name  = "myapp"
    count = 3    # number mixed with string – ERROR
    enabled = true # bool mixed with string – ERROR
  })
}

# Correct – all strings
locals {
  config = {
    name  = "myapp"
    count = tostring(3) # Convert to string
    enabled = tostring(true) # Convert to string
  }
}

# Or use typed object instead of map
locals {
  app_config = {
    name  = "myapp"
    count = 3
    enabled = true
  }
}
```

CI/CD & Automation

Errors #086–#090

#086 Error: Interactive approval required – use -auto-approve to skip

Root Cause Analysis (RCA)

Terraform requires interactive confirmation before applying changes in non-interactive terminals (CI/CD pipelines). This is a safety feature, not an error, but it causes pipelines to hang.

Solution

Use -auto-approve flag for automated pipelines. Implement a plan+apply workflow where the plan is reviewed separately, and apply runs with -auto-approve only after approval.

Example

```
# CI/CD pipeline pattern
steps:
- name: Terraform Init
  run: terraform init -input=false

- name: Terraform Plan
  run: terraform plan -input=false -out=tfplan

- name: Review Plan # Manual approval step
  uses: trstringer/manual-approval@v1

- name: Terraform Apply
  run: terraform apply -auto-approve tfplan
  # OR: terraform apply -input=false tfplan # Still prompts without -auto-approve
```

#087 Error: Terraform exited with code 1 – plan failed due to unresolvable errors

Root Cause Analysis (RCA)

Terraform exits with code 1 for hard errors (config errors, provider failures) and code 2 for successful plan with changes. CI/CD pipelines treating exit code 2 as failure will incorrectly fail on valid plans.

Solution

Handle Terraform exit codes properly in scripts: 0 = success/no changes, 1 = error, 2 = success with changes. Use set +e or trap to handle exit code 2 appropriately.

Example

```
#!/bin/bash
set +e # Don't exit on non-zero exit codes

terraform plan -detailed-exitcode -out=tfplan
PLAN_EXIT_CODE=$?

case $PLAN_EXIT_CODE in
  0) echo "No changes needed"; exit 0 ;;
  1) echo "Plan failed!"; exit 1 ;;
```

```
2) echo "Changes detected – proceeding to apply" ;;
esac

# Apply changes
terraform apply tfplan
if [ $? -ne 0 ]; then
    echo "Apply failed!"
    exit 1
fi
```

#088 Error: Error loading state: AccessDenied – Access Denied to S3 bucket

Root Cause Analysis (RCA)

The IAM principal running Terraform doesn't have `s3:GetObject` or `s3:PutObject` permissions on the state bucket. This often occurs in cross-account setups or after IAM policy changes.

Solution

Grant the required S3 permissions to the Terraform execution role. Use resource-based policies on the S3 bucket in addition to identity-based policies for cross-account access.

Example

```
# IAM policy for Terraform state access
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject",
        "s3:DeleteObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3:::my-tf-state-bucket",
        "arn:aws:s3:::my-tf-state-bucket/*"
      ]
    },
    {
      "Effect": "Allow",
      "Action": [
        "dynamodb:GetItem",
        "dynamodb:PutItem",
        "dynamodb:DeleteItem"
      ],
      "Resource": "arn:aws:dynamodb:*:*:table/terraform-state-locks"
    }
  ]
}
```

#089 Error: Error running Terraform in parallel – race condition on shared state

Root Cause Analysis (RCA)

Multiple Terraform jobs are running simultaneously against the same state file without proper locking. This causes state corruption as concurrent writes can overwrite each other.

Solution

Always use a state backend with locking (S3+DynamoDB, Terraform Cloud). Use pipeline serialization to prevent concurrent runs on the same workspace. Separate workspaces for parallel environment deployments.

Example

```
# Proper backend with locking
terraform {
  backend "s3" {
    bucket     = "tf-state-prod"
    key        = "infrastructure/terraform.tfstate"
    region     = "us-east-1"
    dynamodb_table = "tf-state-lock" # Required for locking
    encrypt    = true
  }
}

# Parallel deployments – separate state per environment
# prod.tfstate, staging.tfstate, dev.tfstate
# Each runs independently with its own lock
```

#090 Error: Terraform version mismatch in CI – different version than local development

Root Cause Analysis (RCA)

The CI/CD pipeline uses a different Terraform version than local development. This causes plan output differences and can lead to state compatibility issues.

Solution

Pin the Terraform version using `required_version` and use `tfenv` or a Docker image to ensure consistent versions across all environments. Store the version in `.terraform-version` file.

Example

```
# versions.tf
terraform {
  required_version = "= 1.7.0" # Exact pin for reproducibility
}

# .terraform-version for tfenv
echo "1.7.0" > .terraform-version

# Docker-based CI/CD for consistency
# Dockerfile
FROM hashicorp/terraform:1.7.0
WORKDIR /workspace
COPY . .
RUN terraform init
CMD ["terraform", "plan"]

# GitHub Actions
- uses: hashicorp/setup-terraform@v3
  with:
    terraform_version: "1.7.0"
```

Tags & Metadata

Errors #091–#092

#091 Error: TagValueInvalid – Tag value exceeds maximum length of 256 characters

Root Cause Analysis (RCA)

An AWS resource tag value exceeds the 256-character limit, or a tag key exceeds the 128-character limit. This often occurs when interpolating long strings or JSON data into tags.

Solution

Truncate tag values to stay within the 256-character limit for values and 128 for keys. Avoid storing structured data in tags; use DynamoDB or SSM for complex metadata.

Example

```
locals {
  # Truncate long values safely
  commit_sha = substr(var.git_commit_sha, 0, 40)
  branch_name = substr(var.git_branch, 0, 100)

  common_tags = {
    Environment = var.environment # Short
    Project     = var.project_name # Short
    ManagedBy   = "Terraform"
    GitCommit   = local.commit_sha
    Branch      = local.branch_name
  }
}

resource "aws_instance" "web" {
  ami          = data.aws_ami.ubuntu.id
  instance_type = "t3.micro"
  tags         = local.common_tags
}
```

#092 Error: InvalidTag.Reserved – Tags starting with 'aws:' are reserved

Root Cause Analysis (RCA)

A tag with the 'aws:' prefix was added in the configuration. AWS reserves all tags beginning with 'aws:' for internal use and doesn't allow users to create them.

Solution

Remove or rename tags that start with 'aws:'. Use a different prefix like 'app:' or your organization's prefix for custom metadata tags.

Example

```
# Wrong – reserved prefix
resource "aws_s3_bucket" "bad" {
  bucket = "my-bucket"
  tags = {
```

```
"aws:Environment" = "production" # RESERVED – not allowed
}
}

# Correct – custom prefix
resource "aws_s3_bucket" "good" {
  bucket = "my-bucket"
  tags = {
    "app:Environment" = "production"
    "org:Team"        = "platform"
    "Environment"     = "production" # No prefix also valid
  }
}
```

Drift & Refresh

Errors #093–#094

#093 Error: Object has changed outside of Terraform – changes were made outside Terraform

Root Cause Analysis (RCA)

Resources managed by Terraform were modified directly through the cloud console, AWS CLI, or other tools. Terraform detected a difference between the state file and actual resource state (drift).

Solution

Run terraform refresh to update the state, then run terraform plan to see the changes. Decide whether to: (a) accept the manual changes with ignore_changes, (b) revert by applying Terraform config, or (c) import and update config.

Example

```
# Refresh state from real infrastructure
terraform refresh

# Then plan to see diff between desired config and current state
terraform plan

# To ignore specific manual changes (e.g., user-managed tags)
resource "aws_instance" "web" {
  ami      = "ami-xxx"
  instance_type = "t3.micro"

  lifecycle {
    ignore_changes = [
      tags["LastDeployedBy"], # Ignore this tag (set by another process)
      user_data,              # Ignore user_data changes
    ]
  }
}
```

#094 Error: Error refreshing state – resource no longer exists

Root Cause Analysis (RCA)

During terraform refresh or plan, Terraform tries to read a resource's current state from the provider but finds it no longer exists. The resource was deleted outside of Terraform.

Solution

Remove the resource from Terraform state using terraform state rm. If the resource should be recreated, ensure it's still in the configuration and run terraform apply.

Example

```
# Remove deleted resource from state
terraform state rm aws_instance.old_server

# Bulk remove if many resources are gone
terraform state list | grep "old_" | while read addr; do
```



```
terraform state rm "$addr"  
done  
  
# Verify state is clean  
terraform plan # Should show resources to create, not "missing" errors
```

Complex Scenarios & Advanced

Errors #095–#150

#095 Error: Cycle detected – mutual dependency between resources A and B

Root Cause Analysis (RCA)

Two or more resources reference each other's attributes, creating a circular dependency. Terraform cannot determine which resource to create first.

Solution

Break the cycle by extracting the shared dependency. Use placeholder values where needed, create resources in stages, or use external data sources to resolve the circular dependency.

Example

```
# Breaking a cycle between SG and IAM role
# Wrong: SG references role, role references SG

# Correct: Create separate baseline SG
resource "aws_security_group" "base" {
  name = "base-sg"
}

resource "aws_iam_role" "app" {
  name = "app-role"
  # References base SG, not the full SG
}

resource "aws_security_group" "app" {
  name = "app-sg"
  # References IAM role
  tags = { Role = aws_iam_role.app.name }
}
```

#096 Error: Provider produced inconsistent final plan – provider returned different plan after modifications

Root Cause Analysis (RCA)

The provider plugin returned different planned values after Terraform's normalization step. This indicates a bug or unexpected behavior in the provider, often related to computed default values.

Solution

This is typically a provider bug. Update to the latest provider version, report to the provider maintainers, or add `ignore_changes` for the affected attributes as a temporary workaround.

Example

```
# Workaround using ignore_changes
resource "aws_cloudwatch_log_group" "app" {
  name = "/aws/lambda/myfunction"

  lifecycle {
```

```

ignore_changes = [
  # Provider sometimes normalizes this differently
  retention_in_days
]
}
}

# Also try upgrading the provider
terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.31" # Use latest stable version
    }
  }
}
}

```

#097 Error: Invalid resource type – resource type 'X' does not exist in provider

Root Cause Analysis (RCA)

The resource type referenced in the configuration doesn't exist in the installed provider version. This can happen after provider upgrades that removed or renamed resource types, or when there's a typo.

Solution

Check the provider CHANGELOG for renamed or removed resource types. Update the resource type name in your configuration. Use the correct provider version that supports the resource type.

Example

```

# Check available resource types
terraform providers schema -json | \
jq '.provider_schemas["registry.terraform.io/hashicorp/aws"].resource_schemas | keys[]' | \
grep "s3"

# AWS S3 resources were split in provider v4+
# Old: aws_s3_bucket (with many inline blocks)
# New: Separate resources
resource "aws_s3_bucket" "main" { bucket = "my-bucket" }
resource "aws_s3_bucket_versioning" "main" {
  bucket = aws_s3_bucket.main.id
  versioning_configuration { status = "Enabled" }
}

```

#098 Error: Error waiting for resource – error reading resource after create: 404

Root Cause Analysis (RCA)

A resource was created successfully, but when Terraform immediately tried to read it back to confirm creation, the eventual consistency of the cloud provider API returned a 404. The resource exists but isn't immediately visible.

Solution

Add retry logic or timeouts to handle eventual consistency. Some providers have built-in waiter retry mechanisms. Adding a `time_sleep` resource can help in rare cases.

Example

```

resource "time_sleep" "wait_for_resource" {
  depends_on = [aws_iam_role.app]
}

```

```
    create_duration = "10s" # Wait for IAM propagation
  }

resource "aws_lambda_function" "app" {
  depends_on = [time_sleep.wait_for_resource]
  role       = aws_iam_role.app.arn
  # ...
}

# In versions.tf
terraform {
  required_providers {
    time = {
      source = "hashicorp/time"
      version = "~> 0.9"
    }
  }
}
```

#099 Error: Error serializing JSON – unexpected number type

Root Cause Analysis (RCA)

A number value that is too large for JSON standard integer representation, or contains special values like Infinity or NaN, is being encoded in a JSON policy document.

Solution

Use appropriate number types and ranges. Convert large integers to strings in policies. Avoid special floating-point values in JSON contexts.

Example

```
# Use strings for large numbers in ARNs
data "aws_iam_policy_document" "access" {
  statement {
    actions = ["s3:GetObject"]
    resources = ["${aws_s3_bucket.data.arn}/*"]
  }
  statement {
    actions = ["kms:Decrypt"]
    resources = [aws_kms_key.main.arn]
    condition {
      test = "StringEquals"
      variable = "kms:CallerAccount"
      # Account ID is a 12-digit number – use string!
      values = [data.aws_caller_identity.current.account_id]
    }
  }
}
```

#100 Error: Plugin crash – the Terraform provider plugin exited unexpectedly

Root Cause Analysis (RCA)

The provider plugin process crashed due to a bug, memory issue, or encountering an unexpected state. This is typically a provider bug rather than a user configuration error.

Solution

Enable provider logging (TF_LOG=DEBUG) to capture details before the crash. Update to the latest provider version. Report the issue to the provider maintainers with the crash log. Try running the failing operation in isolation.

Example

```
# Enable debug logging to capture crash details
export TF_LOG=DEBUG
export TF_LOG_PATH="/tmp/terraform-debug.log"
terraform apply 2>&1 | tee /tmp/terraform-apply.log

# Try with just the failing resource
terraform apply -target=aws_resource.failing

# Update provider version
terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0" # Update to latest
    }
  }
}
terraform init -upgrade
```

#101 Error: Invalid attribute combination – attributes 'X' and 'Y' are mutually exclusive

Root Cause Analysis (RCA)

The resource configuration specifies two attributes that cannot be used together. The provider schema enforces that only one of these may be set at a time.

Solution

Choose only one of the mutually exclusive attributes. Read the provider documentation carefully to understand which attributes conflict and which are alternatives.

Example

```
# Wrong – both specified
resource "aws_db_instance" "bad" {
  db_subnet_group_name = aws_db_subnet_group.main.name
  publicly_accessible   = true # Can conflict with subnet group in some configs
  # availability_zone conflicts with multi_az
  multi_az             = true
  availability_zone     = "us-east-1a" # WRONG – mutually exclusive with multi_az
}

# Correct
resource "aws_db_instance" "good" {
  db_subnet_group_name = aws_db_subnet_group.main.name
  multi_az             = true # No availability_zone when multi_az = true
}
```

#102 Error: Invalid attribute value – attribute 'X' must be one of: [list of valid values]

Root Cause Analysis (RCA)

An attribute's value doesn't match the set of accepted enumeration values. The provider schema enforces specific allowed values for certain attributes.

Solution

Use one of the documented valid values. Check the provider documentation for the exact accepted values, as they are case-sensitive.

Example

```
# Wrong – incorrect protocol value
resource "aws_security_group_rule" "bad" {
  protocol = "TCP" # Wrong – must be lowercase
}

# Correct
resource "aws_security_group_rule" "good" {
  type      = "ingress"
  from_port = 443
  to_port   = 443
  protocol  = "tcp" # Correct: lowercase
  cidr_blocks = ["0.0.0.0/0"]
  security_group_id = aws_security_group.web.id
}

# For all traffic
resource "aws_security_group_rule" "all" {
  protocol = "-1" # -1 means all protocols
  from_port = 0
  to_port = 0
}
```

#103 Error: Resource address is ambiguous – multiple resources match the given address

Root Cause Analysis (RCA)

Using terraform target, state mv, or state rm with a partial address that matches multiple resources. This prevents Terraform from determining which specific resource to operate on.

Solution

Use a fully qualified resource address including the module path and index. Run terraform state list to see exact addresses.

Example

```
# Ambiguous – could match multiple modules
# terraform state rm aws_instance.web # Which module?

# Exact addresses
terraform state list | grep aws_instance

# Use fully qualified paths
terraform state rm 'module.prod.aws_instance.web[0]'
terraform state rm 'module.staging.aws_instance.web[0]'

# Target specific resource
terraform apply -target='module.prod.aws_instance.web[0]'
```

#104 Error: Insufficient role permissions – cannot assume role 'arn:aws:iam::X:role/Y'

Root Cause Analysis (RCA)

Terraform is trying to assume an IAM role for cross-account or elevated permissions, but the current principal lacks sts:AssumeRole permission, or the role's trust policy doesn't allow this principal.

Solution

Add sts:AssumeRole permission to the current IAM user/role, and ensure the target role's trust policy allows the source principal. Check for conditions in the trust policy that might block assumption.

Example

```
# Provider with role assumption
provider "aws" {
  region = "us-east-1"
  assume_role {
    role_arn    = "arn:aws:iam::TARGET_ACCOUNT_ID:role/TerraformRole"
    session_name = "TerraformSession"
    external_id = var.external_id # For extra security
  }
}

# Target role's trust policy
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {
      "AWS": "arn:aws:iam::SOURCE_ACCOUNT:root"
    },
    "Action": "sts:AssumeRole",
    "Condition": {
      "StringEquals": { "sts:ExternalId": "unique-external-id" }
    }
  }]
}
```

#105 Error: Error creating Lambda Function – CodeStorageExceedException

Root Cause Analysis (RCA)

The total deployment package size (Lambda function code + layers) stored in S3 for your account in that region has reached the 75 GB limit.

Solution

Delete unused Lambda function versions, remove old layers, and clean up unreferenced code from the Lambda service bucket. Implement version retention policies.

Example

```
resource "aws_lambda_function" "app" {
  function_name = "my-function"
  role          = aws_iam_role.lambda.arn
  handler       = "index.handler"
  runtime       = "nodejs20.x"
  filename      = data.archive_file.lambda.output_path

  # Limit concurrent executions
  reserved_concurrent_executions = 100
}

# Clean up old versions with AWS CLI
aws lambda list-versions-by-function \
  --function-name my-function \
  --query 'Versions[?Version!=`$LATEST`].[Version]' \
  --output text | \
while read version; do
```

```
aws lambda delete-function --function-name my-function --qualifier "$version"
done
```

#106 Error: Error updating ECS service – ServiceNotActiveException

Root Cause Analysis (RCA)

An ECS service is being modified while it's in a non-active state (being deleted, in a failed state, or being drained). Terraform can't update services that aren't in ACTIVE state.

Solution

Wait for the service to return to ACTIVE state, or recreate it. Check the ECS console for service events to diagnose why the service isn't active.

Example

```
resource "aws_ecs_service" "app" {
  name           = "my-service"
  cluster        = aws_ecs_cluster.main.id
  task_definition = aws_ecs_task_definition.app.arn
  desired_count  = 2

  deployment_minimum_healthy_percent = 50
  deployment_maximum_percent         = 200

  lifecycle {
    # Ignore task definition changes during rolling updates
    ignore_changes = [task_definition]
  }

  timeouts {
    create = "10m"
    update = "10m"
    delete = "10m"
  }
}
```

#107 Error: invalid Terraform address – address cannot contain spaces

Root Cause Analysis (RCA)

A resource address used in commands like terraform state mv or terraform import contains spaces, which aren't valid in resource addresses. Resource names must use underscores or hyphens.

Solution

Replace spaces in resource names with underscores in your Terraform configuration. Resource names follow the pattern: type.name where name can only contain letters, digits, underscores, and hyphens.

Example

```
# Wrong – spaces not allowed in resource names
resource "aws_instance" "web server" { } # INVALID

# Correct – use underscores
resource "aws_instance" "web_server" {
  ami           = "ami-xxx"
  instance_type = "t3.micro"
  tags = {
    Name = "Web Server" # Display name can have spaces in tags
  }
}
```



```

}
}

# Valid resource name patterns
# web_server, webServer, web-server, web_server_01

```

#108 Error: Cannot destroy resource – termination protection is enabled

Root Cause Analysis (RCA)

EC2 instances, RDS databases, and other resources have termination protection enabled. This is an API-level protection that prevents accidental deletion, separate from Terraform's `prevent_destroy` lifecycle.

Solution

Disable termination protection before destroying. This can be done through AWS Console, CLI, or by setting the appropriate Terraform attribute to false and applying before destroying.

Example

```

# EC2 termination protection
resource "aws_instance" "prod" {
  ami      = data.aws_ami.ubuntu.id
  instance_type = "t3.large"

  disable_api_termination = true # Enable protection for prod

  lifecycle {
    prevent_destroy = true # Double protection in Terraform
  }
}

# To destroy: first disable protection
# aws ec2 modify-instance-attribute --instance-id i-xxx --no-disable-api-termination
# OR update disable_api_termination = false and apply, then destroy

```

#109 Error: Error reading SSM Parameter – ParameterNotFound

Root Cause Analysis (RCA)

A data source or resource references an SSM Parameter that doesn't exist. The parameter name may be incorrect, in the wrong region, or doesn't exist in the account.

Solution

Verify the parameter name and region. Use `aws ssm get-parameter` to confirm existence. Consider using `try()` with the data source to handle missing parameters gracefully.

Example

```

# Verify parameter exists
aws ssm get-parameter \
  --name "/myapp/prod/db-password" \
  --with-decryption

# Create parameters with Terraform
resource "aws_ssm_parameter" "db_password" {
  name      = "/myapp/${var.environment}/db-password"
  description = "Database password for ${var.environment}"
  type      = "SecureString"
  value     = var.db_password
}

```

```
key_id    = aws_kms_key.main.arn

tags = local.common_tags
}

# Reference with data source
data "aws_ssm_parameter" "db_password" {
  name      = "/myapp/${var.environment}/db-password"
  with_decryption = true
  depends_on = [aws_ssm_parameter.db_password]
}
```

#110 Error: Error creating CloudFront distribution – InvalidViewerCertificate

Root Cause Analysis (RCA)

The ACM certificate referenced in the CloudFront distribution isn't in the us-east-1 region. CloudFront requires all ACM certificates to be in us-east-1 (Virginia) regardless of where the distribution serves content.

Solution

Create or request the ACM certificate specifically in us-east-1 using a provider alias. Use a separate provider configuration for us-east-1 even if your infrastructure is in another region.

Example

```
# Must use us-east-1 for CloudFront certificates
provider "aws" {
  alias = "us_east_1"
  region = "us-east-1"
}

resource "aws_acm_certificate" "cdn" {
  provider      = aws.us_east_1 # MUST be us-east-1 for CloudFront
  domain_name   = "cdn.example.com"
  validation_method = "DNS"
}

resource "aws_cloudfront_distribution" "main" {
  viewer_certificate {
    acm_certificate_arn = aws_acm_certificate.cdn.arn
    ssl_support_method = "sni-only"
  }
  # ...
}
```

#111 Error: Error applying plan – provider configuration was not present during destroy

Root Cause Analysis (RCA)

A resource was created using a provider with an alias, but during terraform destroy, the provider alias configuration is missing. This commonly happens in multi-region setups.

Solution

Ensure all required provider configurations (including aliases) are present when running destroy operations. Keep provider configurations in dedicated files and don't conditionally include them.

Example

```
# providers.tf – keep all providers always available
provider "aws" {
  region = "us-east-1"
}

provider "aws" {
  alias = "us_west"
  region = "us-west-2"
}

provider "aws" {
  alias = "eu_west"
  region = "eu-west-1"
}

# Even during destroy, these providers must exist
# to destroy resources that were created with them
```

#112 Error: Error creating ALB Listener Rule – TargetGroupAssociationLimit exceeded

Root Cause Analysis (RCA)

An ALB listener has reached the maximum number of rules (100 per listener). Additional rules cannot be added until existing rules are removed.

Solution

Consolidate routing rules using path patterns and host-based routing. Consider using multiple listeners, or deploy a second ALB for additional services. Review and remove unused rules.

Example

```
# Consolidate rules using priority
resource "aws_lb_listener_rule" "services" {
  for_each = var.service_routes # Map of path -> target group ARN

  listener_arn = aws_lb_listener.https.arn
  priority     = each.value.priority

  condition {
    path_pattern {
      values = each.value.paths # Multiple paths per rule
    }
  }

  action {
    type      = "forward"
    target_group_arn = each.value.target_group_arn
  }
}

# Instead of one rule per path, group related paths
variable "service_routes" {
  default = {
    api = { paths = ["/api/*", "/api/v1/*", "/api/v2/*"], priority = 10, target_group_arn = "..." }
    admin = { paths = ["/admin/*", "/admin/api/*"], priority = 20, target_group_arn = "..." }
  }
}
```

#113 Error: Error deleting S3 Bucket – BucketNotEmpty

Root Cause Analysis (RCA)

Terraform is trying to delete an S3 bucket that still contains objects (and possibly versions). AWS requires buckets to be empty before deletion.

Solution

Enable `force_destroy = true` on the bucket resource to automatically delete all objects. For production buckets, manually empty them first. Be cautious: `force_destroy` permanently deletes all objects.

Example

```
# Development/test buckets – auto-empty on destroy
resource "aws_s3_bucket" "dev" {
  bucket     = "myapp-dev-data"
  force_destroy = true # OK for dev
}

# Production – prevent accidental deletion
resource "aws_s3_bucket" "prod" {
  bucket = "myapp-prod-data"
  # force_destroy = false (default)
}

# Manually empty before destroy
aws s3 rm s3://myapp-prod-data --recursive
aws s3api delete-objects \
  --bucket myapp-prod-data \
  --delete "$(aws s3api list-object-versions \
    --bucket myapp-prod-data \
    --output json \
    --query 'Objects: Versions[].{Key: Key, VersionId: VersionId}')"
terraform destroy -target=aws_s3_bucket.prod
```

#114 Error: Error creating DynamoDB table – ResourceInUseException

Root Cause Analysis (RCA)

A DynamoDB table with the same name already exists and is in CREATING or DELETING state, or another operation (like adding a GSI) is in progress on an existing table.

Solution

Wait for the existing operation to complete. If the table is in a stuck state, check the DynamoDB console. Use unique table names per environment to prevent conflicts.

Example

```
resource "aws_dynamodb_table" "app" {
  name         = "myapp-${var.environment}-data"
  billing_mode = "PAY_PER_REQUEST"
  hash_key     = "id"
  range_key    = "timestamp"

  attribute {
    name = "id"
    type = "S"
  }
  attribute {
    name = "timestamp"
    type = "N"
  }
}
```

```
ttl {
  attribute_name = "ttl"
  enabled       = true
}

tags = local.common_tags
}
```

#115 Error: Error creating AutoScaling Group – AlreadyExists

Root Cause Analysis (RCA)

An Auto Scaling Group with the same name already exists. ASG names must be unique within an account and region. This often occurs after a failed destroy or when names are hardcoded.

Solution

Use unique ASG names incorporating environment and a random suffix. Clean up the existing ASG via the console if it's orphaned. Use terraform import to bring it under management.

Example

```
resource "random_pet" "asg_name" {
  length  = 2
  separator = "-"
}

resource "aws_autoscaling_group" "app" {
  name = "asg-${var.environment}-${random_pet.asg_name.id}"

  min_size      = var.min_instances
  max_size      = var.max_instances
  desired_capacity = var.desired_instances

  vpc_zone_identifier = aws_subnet.private[*].id
  target_group_arns   = [aws_lb_target_group.app.arn]

  launch_template {
    id      = aws_launch_template.app.id
    version = "$Latest"
  }
}
```

#116 Error: Error creating CloudWatch Alarm – InvalidParameterCombination

Root Cause Analysis (RCA)

A CloudWatch alarm is configured with an incompatible combination of parameters. Common issues include using ExtendedStatistics and Statistics together, or specifying DatapointsToAlarm greater than EvaluationPeriods.

Solution

Review CloudWatch alarm parameter constraints. DatapointsToAlarm must be $\leq$ EvaluationPeriods. Use either statistic OR extended_statistic, not both.

Example

```
resource "aws_cloudwatch_metric_alarm" "cpu_high" {
  alarm_name      = "high-cpu-${var.environment}"
  comparison_operator = "GreaterThanThreshold"
```

```

evaluation_periods = 5
datapoints_to_alarm = 3 # Must be <= evaluation_periods (3 of 5)
metric_name        = "CPUUtilization"
namespace          = "AWS/EC2"
period             = 60
statistic           = "Average" # Use either statistic OR extended_statistic
# extended_statistic = "p99" # DON'T use both
threshold           = 80

alarm_actions = [aws_sns_topic.alerts.arn]
ok_actions    = [aws_sns_topic.alerts.arn]
}

```

#117 Error: Error reading Route53 record – InvalidInput

Root Cause Analysis (RCA)

A Route 53 DNS record configuration has invalid values, such as an invalid record type, an FQDN without a trailing dot, or an alias pointing to an endpoint in an unsupported region.

Solution

Ensure DNS names end with a trailing dot when using FQDNs. Verify the record type is valid. For alias records, confirm the target is in a supported region and the health check settings are correct.

Example

```

# Route 53 records – always include trailing dot in FQDNs
resource "aws_route53_record" "www" {
  zone_id = aws_route53_zone.main.zone_id
  name     = "www.example.com" # Trailing dot added automatically
  type     = "A"

  alias {
    name           = aws_lb.main.dns_name
    zone_id        = aws_lb.main.zone_id
    evaluate_target_health = true
  }
}

# CNAME record
resource "aws_route53_record" "api" {
  zone_id = aws_route53_zone.main.zone_id
  name     = "api"
  type     = "CNAME"
  ttl      = 300
  records  = ["api.example.com."] # Trailing dot for FQDN
}

```

#118 Error: Error creating WAF WebACL – WAFNonexistentItemException for rule

Root Cause Analysis (RCA)

A WAF WebACL references a rule group, IP set, or regex pattern set that doesn't exist yet. Terraform may not have created the dependency first, or the ARN is incorrect.

Solution

Ensure all WAF rules, IP sets, and managed rule groups referenced in a WebACL are created before the WebACL. Use explicit `depends_on` or reference them directly to establish dependencies.

Example

```
resource "aws_wafv2_ip_set" "blocklist" {
  name      = "ip-blocklist"
  scope     = "REGIONAL"
  ip_address_version = "IPV4"
  addresses = var.blocked_ips
}

resource "aws_wafv2_web_acl" "main" {
  name = "main-waf"
  scope = "REGIONAL"

  default_action { allow {} }

  rule {
    name     = "BlockBadIPs"
    priority = 1

    action { block {} }

    statement {
      ip_set_reference_statement {
        arn = aws_wafv2_ip_set.blocklist.arn # Implicit dependency
      }
    }

    visibility_config {
      cloudwatch_metrics_enabled = true
      metric_name                 = "BlockBadIPs"
      sampled_requests_enabled   = true
    }
  }
}
```

#119 Error: Error creating Cognito User Pool – InvalidParameterException for schema**Root Cause Analysis (RCA)**

A Cognito User Pool schema attribute has an invalid configuration, such as an attribute name that's reserved, attribute constraints that are too restrictive, or a mutable attribute marked as immutable after creation.

Solution

Review Cognito attribute naming constraints. Standard attributes like email and phone_number have specific configuration requirements. Custom attributes must be prefixed with custom: in the API but not in Terraform.

Example

```
resource "aws_cognito_user_pool" "main" {
  name = "myapp-users"

  username_attributes = ["email"]
  auto_verified_attributes = ["email"]

  password_policy {
    minimum_length      = 12
    require_lowercase   = true
    require_numbers     = true
    require_symbols     = true
    require_uppercase   = true
  }
}
```

```

schema {
  name          = "email"
  attribute_data_type = "String"
  mutable       = true
  required      = true
  string_attribute_constraints {
    min_length = 5
    max_length = 256
  }
}

# Custom attributes – no custom: prefix in Terraform
schema {
  name          = "department"
  attribute_data_type = "String"
  mutable       = true
  required      = false
}
}

```

#120 Error: Error creating EKS Cluster – UnsupportedAvailabilityZoneException

Root Cause Analysis (RCA)

An EKS cluster is being created with a subnet in an availability zone that doesn't support the EKS control plane. Not all AZs support EKS in every region.

Solution

Check which AZs support EKS using the AWS CLI. Use EKS-supported AZs for control plane subnets. Create an EKS cluster without specifying subnets that are in unsupported AZs.

Example

```

# Check supported AZs for EKS
aws eks describe-addon-versions --region us-east-1 | \
jq '.addons[].addonVersions[0].compatibilities[].platformVersions[]'

# Use data source to filter supported AZs
data "aws_availability_zones" "available" {
  filter {
    name = "opt-in-status"
    values = ["opt-in-not-required"]
  }
}

resource "aws_eks_cluster" "main" {
  name     = "prod-cluster"
  role_arn = aws_iam_role.eks.arn

  vpc_config {
    subnet_ids = [
      # Use subnets in recommended AZs
      aws_subnet.private["us-east-1a"].id,
      aws_subnet.private["us-east-1b"].id,
      aws_subnet.private["us-east-1c"].id,
    ]
  }
}

```


#121 Error: insufficient permissions to list provider schema**Root Cause Analysis (RCA)**

The Terraform configuration has provider blocks that require specific initialization permissions. The credentials provided cannot retrieve provider schema information needed for validation.

Solution

Run terraform init with credentials that have at least read-only access. The init phase requires provider download permissions but not necessarily resource management permissions.

Example

```
# Use separate credentials for init vs apply
# init-credentials.sh
export AWS_ACCESS_KEY_ID="$READONLY_KEY_ID"
export AWS_SECRET_ACCESS_KEY="$READONLY_SECRET"
terraform init

# apply-credentials.sh
export AWS_ACCESS_KEY_ID="$ADMIN_KEY_ID"
export AWS_SECRET_ACCESS_KEY="$ADMIN_SECRET"
terraform apply

# Or use profiles
terraform init # Uses default profile (read-only)
AWS_PROFILE=admin terraform apply
```

#122 Error: Error creating SQS Queue – InvalidAttributeValue for RedrivePolicy**Root Cause Analysis (RCA)**

An SQS queue's redrive policy (dead-letter queue configuration) references an SQS queue ARN that's in a different AWS account without proper permissions, or the maxReceiveCount is outside the valid range (1-1000).

Solution

Ensure the dead-letter queue is in the same region as the source queue. Cross-account DLQ references require additional permissions. Set maxReceiveCount between 1 and 1000.

Example

```
resource "aws_sqs_queue" "dlq" {
  name = "myapp-${var.environment}-dlq"
}

resource "aws_sqs_queue" "main" {
  name                = "myapp-${var.environment}-queue"
  visibility_timeout_seconds = 30
  message_retention_seconds = 86400

  redrive_policy = jsonencode({
    deadLetterTargetArn = aws_sqs_queue.dlq.arn
    maxReceiveCount     = 5 # Must be between 1-1000
  })
}

resource "aws_sqs_queue_redrive_allow_policy" "dlq" {
  queue_url = aws_sqs_queue.dlq.id

  redrive_allow_policy = jsonencode({
```

```
redrivePermission = "byQueue"
sourceQueueArns = [aws_sqs_queue.main.arn]
})
}
```

#123 Error: Error creating SNS Subscription – InvalidParameter

Root Cause Analysis (RCA)

An SNS subscription has an invalid endpoint URL, an incompatible protocol/endpoint combination, or missing confirmation for HTTP/HTTPS subscriptions. Each protocol has specific endpoint format requirements.

Solution

Verify the endpoint format matches the protocol. SQS endpoints use queue ARNs, Lambda uses function ARNs, email uses email addresses. HTTP/HTTPS subscriptions need confirmation which Terraform handles automatically.

Example

```
resource "aws_sns_topic" "alerts" {
  name = "app-alerts-${var.environment}"
}

# SQS subscription
resource "aws_sns_topic_subscription" "sqs" {
  topic_arn = aws_sns_topic.alerts.arn
  protocol = "sqs"
  endpoint = aws_sqs_queue.main.arn
}

# Lambda subscription
resource "aws_sns_topic_subscription" "lambda" {
  topic_arn = aws_sns_topic.alerts.arn
  protocol = "lambda"
  endpoint = aws_lambda_function.processor.arn
}

# Email subscription (requires manual confirmation)
resource "aws_sns_topic_subscription" "email" {
  topic_arn = aws_sns_topic.alerts.arn
  protocol = "email"
  endpoint = "alerts@company.com"
}
```

#124 Error: Error creating Step Functions State Machine – InvalidDefinition

Root Cause Analysis (RCA)

The Step Functions state machine definition JSON is invalid. Common issues include referencing undefined states, invalid task resource ARNs, missing Start and End states, or JSON syntax errors.

Solution

Validate the state machine definition using the AWS Step Functions console or the ASL validator. Use `templatefile()` or `jsonencode()` to construct the definition and validate it before applying.

Example

```
resource "aws_sfn_state_machine" "order_processor" {
  name = "order-processor"
  role_arn = aws_iam_role.sfn.arn
}
```

```

definition = jsonencode({
  Comment = "Order processing workflow"
  StartAt = "ValidateOrder"
  States = {
    ValidateOrder = {
      Type    = "Task"
      Resource = aws_lambda_function.validate.arn
      Next    = "ProcessPayment"
    }
    ProcessPayment = {
      Type    = "Task"
      Resource = aws_lambda_function.payment.arn
      End     = true
    }
  }
})
}

```

#125 Error: Error creating CodePipeline – ValidationException for stages

Root Cause Analysis (RCA)

A CodePipeline definition has an invalid stage configuration. Common issues include duplicate stage names, a Source stage not in first position, missing action configurations, or referencing non-existent CodeBuild projects.

Solution

Ensure the pipeline stages are valid: Source must be first, stages must have unique names, and all referenced resources must exist. Use `depends_on` to ensure CodeBuild projects are created before the pipeline.

Example

```

resource "aws_codepipeline" "app" {
  name       = "myapp-pipeline"
  role_arn   = aws_iam_role.codepipeline.arn

  artifact_store {
    location = aws_s3_bucket.artifacts.bucket
    type     = "S3"
  }

  stage {
    name = "Source" # Must be first stage
    action {
      name      = "Source"
      category  = "Source"
      owner     = "AWS"
      provider  = "CodeStarSourceConnection"
      version   = "1"
      output_artifacts = ["source_output"]
      configuration = {
        ConnectionArn = aws_codestarconnections_connection.github.arn
        FullRepositoryId = "org/repo"
        BranchName      = "main"
      }
    }
  }

  stage {
    name = "Build"
    action {

```

```
name      = "Build"
category  = "Build"
owner     = "AWS"
provider  = "CodeBuild"
version   = "1"
input_artifacts = ["source_output"]
output_artifacts = ["build_output"]
configuration = {
  ProjectName = aws_codebuild_project.app.name
}
```

#126 Error: Error creating ElastiCache Cluster – InvalidParameterCombination – engine version not compatible with node type

Root Cause Analysis (RCA)

The ElastiCache node type and engine version combination is not supported. Newer engine versions may require newer node type generations, and some node families don't support all engine versions.

Solution

Check the ElastiCache documentation for compatible engine version and node type combinations. Use newer node types (r6g, m6g) with newer Redis/Valkey versions.

Example

```
resource "aws_elasticache_cluster" "redis" {
  cluster_id      = "myapp-${var.environment}"
  engine          = "redis"
  engine_version  = "7.1" # Use recent stable version
  node_type       = "cache.r7g.large" # Gen 7 supports Redis 7.x
  num_cache_nodes = 1
  parameter_group_name = "default.redis7"
  port            = 6379
  subnet_group_name = aws_elasticache_subnet_group.main.name

  security_group_ids = [aws_security_group.redis.id]
}
```

#127 Error: Error creating MSK Cluster – InvalidRequestException for broker node group info

Root Cause Analysis (RCA)

An Amazon MSK (Managed Streaming for Kafka) cluster configuration has invalid broker node settings, such as an unsupported instance type, too few brokers for the specified number of AZs, or incompatible Kafka version.

Solution

Ensure the number of brokers is a multiple of the number of AZs. Use MSK-supported instance types. Verify the Kafka version is supported by MSK.

Example

```
resource "aws_msk_cluster" "main" {
  cluster_name      = "myapp-kafka"
  kafka_version     = "3.5.1"
  number_of_broker_nodes = 3 # Must be multiple of AZ count (3 AZs here)
```

```
broker_node_group_info {
  instance_type = "kafka.m5.large"
  client_subnets = [
    aws_subnet.private["us-east-1a"].id,
    aws_subnet.private["us-east-1b"].id,
    aws_subnet.private["us-east-1c"].id,
  ]
  security_groups = [aws_security_group.msk.id]
  storage_info {
    ebs_storage_info { volume_size = 100 }
  }
}

encryption_info {
  encryption_in_transit {
    client_broker = "TLS"
    in_cluster = true
  }
}
}
```

#128 Error: Error creating Glue Job – InvalidInputException for connections

Root Cause Analysis (RCA)

An AWS Glue job configuration references a Glue connection that doesn't exist, or the job type is incompatible with the specified connections or worker type.

Solution

Ensure Glue connections exist before referencing them in jobs. Verify that the worker type and number of workers are compatible with the Glue job type. Use `depends_on` for Glue connections.

Example

```
resource "aws_glue_connection" "rds" {
  name = "rds-connection"
  connection_properties = {
    JDBC_CONNECTION_URL = "jdbc:postgresql://${aws_db_instance.main.endpoint}/mydb"
    USERNAME             = "glue_user"
    PASSWORD             = var.db_password
  }
  physical_connection_requirements {
    subnet_id          = aws_subnet.private[0].id
    security_group_id_list = [aws_security_group.glue.id]
    availability_zone   = "us-east-1a"
  }
}

resource "aws_glue_job" "etl" {
  name          = "myapp-etl"
  role_arn      = aws_iam_role.glue.arn
  glue_version  = "4.0"
  worker_type   = "G.1X"
  number_of_workers = 2

  connections = [aws_glue_connection.rds.name] # Must exist first

  command {
    script_location = "s3://${aws_s3_bucket.scripts.bucket}/etl.py"
    python_version  = "3"
  }
}
```

```
}  
}
```

#129 Error: Error creating Athena Workgroup – InvalidRequestException for configuration

Root Cause Analysis (RCA)

An Athena workgroup configuration has invalid settings, such as a non-existent output S3 bucket, improperly formatted S3 path, or invalid encryption configuration.

Solution

Ensure the S3 output location exists and the path format is correct (s3://bucket-name/prefix/). Create the S3 bucket before the workgroup. Verify encryption key permissions.

Example

```
resource "aws_s3_bucket" "athena_results" {  
  bucket = "myapp-athena-results-${data.aws_caller_identity.current.account_id}"  
}  
  
resource "aws_athena_workgroup" "main" {  
  name = "myapp-analytics"  
  
  configuration {  
    enforce_workgroup_configuration = true  
    publish_cloudwatch_metrics_enabled = true  
  
    result_configuration {  
      output_location = "s3://${aws_s3_bucket.athena_results.bucket}/results/" # Trailing slash required  
  
      encryption_configuration {  
        encryption_option = "SSE_KMS"  
        kms_key_arn       = aws_kms_key.main.arn  
      }  
    }  
  }  
  
  depends_on = [aws_s3_bucket.athena_results]  
}
```

#130 Error: Provider schema iteration failed – the provider returned no schema

Root Cause Analysis (RCA)

During terraform init or plan, the provider plugin failed to return its schema. This can be caused by network issues during provider download, disk space problems, or a corrupted provider binary.

Solution

Delete the .terraform directory and re-run terraform init. Check disk space availability. Verify the provider version exists in the registry. Check for network connectivity issues.

Example

```
# Clear provider cache  
rm -rf .terraform .terraform.lock.hcl  
  
# Check disk space  
df -h ~/.terraform.d/  
df -h .
```

```
# Reinitialize
terraform init

# Verify provider in registry
curl -s "https://registry.terraform.io/v1/providers/hashicorp/aws/versions" | \
jq '.versions[-3:][].version'

# If using custom providers, validate binary
ls -la ~/.terraform.d/plugins/
```

#131 Error: Error creating Kinesis Stream – LimitExceededException – shard limit exceeded

Root Cause Analysis (RCA)

The Kinesis Data Streams shard limit (default 500 shards per region) has been reached. Each new stream's shards count against this regional limit.

Solution

Request a shard limit increase via AWS Service Quotas. Reduce the number of shards in new streams by using on-demand mode or optimizing the number of shards based on throughput requirements.

Example

```
resource "aws_kinesis_stream" "data" {
  name = "myapp-events-${var.environment}"

  # Use ON_DEMAND to avoid manual shard management
  stream_mode_details {
    stream_mode = "ON_DEMAND" # Automatically scales shards
  }

  # Or specify shards for PROVISIONED mode
  # shard_count = 1

  retention_period      = 24 # Hours (24-8760)
  enforce_consumer_deletion = false

  encryption_type = "KMS"
  kms_key_id      = aws_kms_key.main.id
}
```

#132 Error: Error creating EventBridge Rule – InvalidEventPattern

Root Cause Analysis (RCA)

An EventBridge (CloudWatch Events) rule has an invalid event pattern JSON. Common issues include incorrect service/resource names, invalid JSON structure, or referencing unsupported event fields.

Solution

Validate the event pattern using the AWS EventBridge console's Test Pattern feature. Ensure the event pattern JSON is properly formatted and uses valid event field names.

Example

```
resource "aws_cloudwatch_event_rule" "ec2_state_change" {
  name      = "ec2-state-change-${var.environment}"
  description = "Capture EC2 instance state changes"
```

```

event_pattern = jsonencode({
  source = ["aws.ec2"]
  "detail-type" = ["EC2 Instance State-change Notification"]
  detail = {
    state = ["running", "stopped", "terminated"]
  }
})
}

resource "aws_cloudwatch_event_target" "lambda" {
  rule      = aws_cloudwatch_event_rule.ec2_state_change.name
  target_id = "NotifyLambda"
  arn       = aws_lambda_function.notifier.arn
}

```

#133 Error: Error creating AppSync API – BadRequestException for authentication type

Root Cause Analysis (RCA)

An AppSync GraphQL API has an invalid or incompatible authentication configuration. The authentication type and its associated configuration must match exactly, and additional auth providers need specific settings.

Solution

Ensure the authentication type and its configuration block match. API_KEY requires an expiration.

AMAZON_COGNITO_USER_POOLS requires a user pool ARN. Configure openid_connect_config for OPENID_CONNECT.

Example

```

resource "aws_appsync_graphql_api" "main" {
  name           = "myapp-api"
  authentication_type = "AMAZON_COGNITO_USER_POOLS"

  user_pool_config {
    user_pool_id = aws_cognito_user_pool.main.id
    aws_region   = data.aws_region.current.name
    default_action = "ALLOW"
  }

  # Optional: Add API key as secondary auth
  additional_authentication_provider {
    authentication_type = "API_KEY"
  }

  schema = file("${path.module}/schema.graphql")

  log_config {
    cloudwatch_logs_role_arn = aws_iam_role.appsync_logs.arn
    field_log_level         = "ERROR"
  }
}

```

#134 Error: Error creating API Gateway – BadRequestException for integration URI

Root Cause Analysis (RCA)

An API Gateway integration has an invalid URI. Lambda integration URIs must use the specific invocation ARN format, not the function ARN. REST APIs and HTTP APIs have different URI formats.

Solution

Use the correct URI format for the integration type. For Lambda, use the invocation ARN (arn:aws:apigateway:REGION:lambda:path/2015-03-31/functions/FUNCTION_ARN/invocations). For HTTP, use the full URL.

Example

```
resource "aws_api_gateway_integration" "lambda" {
  rest_api_id      = aws_api_gateway_rest_api.main.id
  resource_id      = aws_api_gateway_resource.main.id
  http_method      = aws_api_gateway_method.main.http_method
  integration_http_method = "POST"
  type             = "AWS_PROXY"

  # Correct Lambda integration URI format
  uri = aws_lambda_function.handler.invoke_arn
  # NOT: aws_lambda_function.handler.arn
}

# Allow API Gateway to invoke Lambda
resource "aws_lambda_permission" "api_gateway" {
  statement_id = "AllowAPIGatewayInvoke"
  action       = "lambda:InvokeFunction"
  function_name = aws_lambda_function.handler.function_name
  principal    = "apigateway.amazonaws.com"
  source_arn    = "${aws_api_gateway_rest_api.main.execution_arn}/*/*"
}
```

#135 Error: Error creating ACM certificate validation – DNS records not found

Root Cause Analysis (RCA)

ACM certificate validation via DNS is waiting for the DNS validation records to be created and propagated. The CNAME records must be added to the domain's Route 53 hosted zone before validation succeeds.

Solution

Create the required DNS validation records in Route 53 using the `aws_acm_certificate_validation` resource combined with the certificate's `domain_validation_options`. Allow time for DNS propagation.

Example

```
resource "aws_acm_certificate" "main" {
  domain_name      = "example.com"
  validation_method = "DNS"

  subject_alternative_names = ["*.example.com"]

  lifecycle {
    create_before_destroy = true
  }
}

# Create DNS validation records
resource "aws_route53_record" "cert_validation" {
  for_each = {
    for dvo in aws_acm_certificate.main.domain_validation_options : dvo.domain_name => {
      name    = dvo.resource_record_name
      record  = dvo.resource_record_value
      type    = dvo.resource_record_type
    }
  }

  allow_overwrite = true
  name            = each.value.name
```

```

records      = [each.value.record]
ttl          = 60
type         = each.value.type
zone_id      = aws_route53_zone.main.zone_id
}

resource "aws_acm_certificate_validation" "main" {
  certificate_arn      = aws_acm_certificate.main.arn
  validation_record_fqdns = [for record in aws_route53_record.cert_validation : record.fqdn]

  timeouts {
    create = "10m" # DNS propagation can take several minutes
  }
}

```

#136 Error: Error creating Config Rule – InsufficientPermissionsException

Root Cause Analysis (RCA)

The AWS Config service doesn't have the required IAM permissions to evaluate or remediate resources. AWS Config requires a specific service role with S3, SNS, and resource-describe permissions.

Solution

Create a properly configured AWS Config service role with the AWSConfigRole managed policy, or create a custom role with the necessary permissions. Attach the role to the config delivery channel.

Example

```

resource "aws_iam_role" "config" {
  name = "aws-config-role"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Action = "sts:AssumeRole"
      Effect = "Allow"
      Principal = { Service = "config.amazonaws.com" }
    }]
  })
}

resource "aws_iam_role_policy_attachment" "config" {
  role      = aws_iam_role.config.name
  policy_arn = "arn:aws:iam::aws:policy/service-role/AWSConfigRole"
}

resource "aws_config_delivery_channel" "main" {
  name          = "main"
  s3_bucket_name = aws_s3_bucket.config.bucket
  depends_on    = [aws_config_configuration_recorder.main]
}

resource "aws_config_configuration_recorder" "main" {
  name      = "main"
  role_arn = aws_iam_role.config.arn
}

```

#137 Error: Error creating GuardDuty Detector – the feature already has a detector

Root Cause Analysis (RCA)

Only one GuardDuty detector can be created per AWS account per region. Attempting to create a second detector fails. This often happens when the detector was created manually or by another Terraform workspace.

Solution

Import the existing GuardDuty detector into Terraform state. Get the detector ID from the AWS console or CLI, then use `terraform import`.

Example

```
# Get existing detector ID
aws guardduty list-detectors --region us-east-1

# Import existing detector
terraform import aws_guardduty_detector.main <DETECTOR_ID>

# GuardDuty detector resource
resource "aws_guardduty_detector" "main" {
  enable = true

  datasources {
    s3_logs {
      enable = true
    }
    kubernetes {
      audit_logs {
        enable = true
      }
    }
    malware_protection {
      scan_ec2_instance_with_findings {
        ebs_volumes {
          enable = true
        }
      }
    }
  }
}
```

#138 Error: Error creating IAM role – MalformedPolicyDocument for assume role policy

Root Cause Analysis (RCA)

The IAM role's assume role policy (trust policy) is malformed. Common issues include incorrect service principal format, missing Version field, or using the wrong principal type (AWS vs Service).

Solution

Validate the trust policy structure. Service principals end in `.amazonaws.com`. AWS principals use ARN format. Use the `aws_iam_policy_document` data source for type-safe policy generation.

Example

```
# Use data source for type-safe trust policies
data "aws_iam_policy_document" "lambda_trust" {
  statement {
    effect = "Allow"
    principals {
      type      = "Service"
      identifiers = ["lambda.amazonaws.com"] # Correct service principal
    }
    actions = ["sts:AssumeRole"]
  }
}
```

```

}
}

resource "aws_iam_role" "lambda" {
  name           = "myapp-lambda-role"
  assume_role_policy = data.aws_iam_policy_document.lambda_trust.json
}

# Multiple service principals
data "aws_iam_policy_document" "multi_trust" {
  statement {
    principals {
      type       = "Service"
      identifiers = ["lambda.amazonaws.com", "edgelambda.amazonaws.com"]
    }
    actions = ["sts:AssumeRole"]
  }
}
}

```

#139 Error: Error creating CloudTrail – InsufficientEncryptionPolicyException

Root Cause Analysis (RCA)

The KMS key policy doesn't grant CloudTrail the necessary permissions to encrypt log files. CloudTrail requires specific permissions in the key policy, not just in IAM policies.

Solution

Update the KMS key policy to explicitly grant CloudTrail permissions to GenerateDataKey and Decrypt. The CloudTrail service principal must be listed in the key policy.

Example

```

data "aws_iam_policy_document" "cloudtrail_kms" {
  statement {
    sid    = "Enable IAM Root Permissions"
    effect = "Allow"
    principals {
      type       = "AWS"
      identifiers = ["arn:aws:iam::${data.aws_caller_identity.current.account_id}:root"]
    }
    actions = ["kms:*"]
    resources = ["*"]
  }
  statement {
    sid    = "Allow CloudTrail to encrypt logs"
    effect = "Allow"
    principals {
      type       = "Service"
      identifiers = ["cloudtrail.amazonaws.com"]
    }
    actions = ["kms:GenerateDataKey*", "kms:Decrypt"]
    resources = ["*"]
    condition {
      test     = "StringLike"
      variable = "kms:EncryptionContext:aws:cloudtrail:arn"
      values   = ["arn:aws:cloudtrail:*:${data.aws_caller_identity.current.account_id}:trail/*"]
    }
  }
}

resource "aws_kms_key" "cloudtrail" {

```

```
policy = data.aws_iam_policy_document.cloudtrail_kms.json
}
```

#140 Error: Error creating Backup Plan – InvalidParameterValueException for schedule expression

Root Cause Analysis (RCA)

An AWS Backup plan contains an invalid cron expression. AWS Backup uses cron expressions but requires the specific format: cron(Minutes Hours DayOfMonth Month DayOfWeek Year).

Solution

Use the correct AWS Backup cron format, which is different from standard Unix cron. Include the Year field and use cron() function notation. Test expressions in the AWS Backup console.

Example

```
resource "aws_backup_plan" "main" {
  name = "myapp-backup-plan"

  rule {
    rule_name      = "daily-backup"
    target_vault_name = aws_backup_vault.main.name

    # AWS Backup cron format: cron(min hour dom month dow year)
    schedule = "cron(0 5 * * ? *)" # Daily at 5:00 AM UTC
    # Note: ? is used for DayOfMonth or DayOfWeek (not both)

    start_window      = 60 # Minutes to start backup
    completion_window = 120 # Minutes to complete backup

    lifecycle {
      cold_storage_after = 30 # Days before moving to cold storage
      delete_after       = 90 # Days before deletion
    }
  }
}
```

#141 Error: 'template_file' data source deprecated – use templatefile() function

Root Cause Analysis (RCA)

The template_file data source from the template provider has been deprecated in favor of the built-in templatefile() function introduced in Terraform 0.12. Old configurations may still use this deprecated approach.

Solution

Replace data.template_file with the templatefile() function directly in your resource configurations. Remove the template provider dependency as it's no longer needed.

Example

```
# Old deprecated way
data "template_file" "user_data" {
  template = file("scripts/user_data.sh.tpl")
  vars = {
    db_host = aws_db_instance.main.address
  }
}

resource "aws_instance" "old" {
  user_data = data.template_file.user_data.rendered
}
```

```
}

# Modern way – use templatefile() function
resource "aws_instance" "new" {
  user_data = templatefile("${path.module}/scripts/user_data.sh.tpl", {
    db_host   = aws_db_instance.main.address
    environment = var.environment
    region    = data.aws_region.current.name
  })
}
```

#142 Error: Error creating VPN Connection – VpnConnectionLimitExceeded

Root Cause Analysis (RCA)

The AWS account has reached the default limit of 50 VPN connections per region. Each Site-to-Site VPN connection consumes one unit of this quota.

Solution

Delete unused VPN connections, consolidate with AWS Direct Connect, or request a quota increase. Consider using AWS Transit Gateway to manage multiple connections more efficiently.

Example

```
# Efficient multi-site connectivity with Transit Gateway
resource "aws_ec2_transit_gateway" "main" {
  description = "Central routing hub"
  auto_accept_shared_attachments = "enable"
}

# VPN connections attach to Transit Gateway
resource "aws_vpn_connection" "site_a" {
  customer_gateway_id = aws_customer_gateway.site_a.id
  transit_gateway_id  = aws_ec2_transit_gateway.main.id
  type                = "ipsec.1"

  static_routes_only = true
}

resource "aws_customer_gateway" "site_a" {
  bgp_asn   = 65000
  ip_address = "203.0.113.1"
  type      = "ipsec.1"
}
```

#143 Error: Unsupported block type 'connection' inside resource block

Root Cause Analysis (RCA)

A connection block for SSH/WinRM configuration has been placed directly inside a resource block without a nested provisioner block. Connection blocks must be paired with provisioner blocks.

Solution

Place connection blocks inside provisioner blocks, or at the resource level if you have multiple provisioners. Better yet, migrate from provisioners to cloud-init scripts or AMI baking.

Example

```
resource "aws_instance" "web" {
  ami      = "ami-xxx"
  instance_type = "t3.micro"
  key_name  = aws_key_pair.deployer.key_name

  # Connection at resource level (applies to all provisioners)
  connection {
    type      = "ssh"
    user      = "ubuntu"
    private_key = file("~/ssh/deployer.pem")
    host      = self.public_ip
  }

  provisioner "remote-exec" {
    inline = [
      "sudo apt-get update",
      "sudo apt-get install -y nginx",
    ]
  }
}

# Better: Use user_data instead of provisioners
resource "aws_instance" "better" {
  user_data = file("scripts/setup.sh")
}
```

#144 Error: terraform plan produced no output when changes are expected

Root Cause Analysis (RCA)

The plan command is run against the wrong workspace, wrong state file, or the wrong working directory. No changes appear because Terraform is evaluating a different set of resources than expected.

Solution

Verify the active workspace, correct state backend, and working directory. Use `terraform workspace show` to confirm the current workspace. Run `terraform state list` to verify what's being managed.

Example

```
# Verify current workspace
terraform workspace show
terraform workspace list

# Check state contents
terraform state list

# Verify backend configuration
terraform init -backend-config="key=correct/path/terraform.tfstate"

# Show current state of a specific resource
terraform state show aws_instance.web

# Run with verbose output
TF_LOG=DEBUG terraform plan 2>&1 | head -100
```

#145 Error: Error creating Elastic Beanstalk Application – TooManyApplicationsException

Root Cause Analysis (RCA)

The Elastic Beanstalk application limit (default 75 per region) has been reached. Applications accumulate over time if old ones aren't cleaned up.

Solution

Delete unused Elastic Beanstalk applications and their associated environments. Request a quota increase if needed. Consider migrating to ECS or EKS for more scalable container management.

Example

```
resource "aws_elastic_beanstalk_application" "app" {
  name       = "myapp-${var.environment}"
  description = "MyApp ${var.environment} environment"

  appversion_lifecycle {
    service_role = aws_iam_role.beanstalk.arn
    max_count    = 10    # Keep only 10 versions
    delete_source_from_s3 = true # Clean up S3 artifacts too
  }
}

resource "aws_elastic_beanstalk_environment" "main" {
  name           = "${var.environment}-env"
  application     = aws_elastic_beanstalk_application.app.name
  solution_stack_name = "64bit Amazon Linux 2023 v4.x.x running Docker"
}
```

#146 Error: Error creating FSx File System – FileSystemAlreadyExists

Root Cause Analysis (RCA)

An FSx file system with the same client request token already exists. The client request token is automatically generated from the resource configuration, so identical configurations create this conflict.

Solution

Ensure each FSx file system has a unique configuration. If importing an existing file system, use terraform import. For idempotent creates, change the storage capacity or other distinguishing attributes.

Example

```
resource "aws_fsx_windows_file_system" "main" {
  active_directory_id = aws_directory_service_directory.main.id
  storage_capacity    = 300
  storage_type        = "SSD"
  subnet_ids          = [aws_subnet.private.id]
  throughput_capacity = 1024

  tags = {
    Name       = "myapp-${var.environment}-fsx"
    Environment = var.environment
  }
}

# Import existing file system
# terraform import aws_fsx_windows_file_system.main fs-xxxxxxxxxxxxxxxxxxxx
```

#147 Error: Error creating Transfer Server – ConflictException – endpoint already exists

Root Cause Analysis (RCA)

An AWS Transfer Family server with a VPC endpoint already exists in the VPC, or a custom hostname (Route 53 record) conflicts with an existing entry. Each server type combination has unique constraints.

Solution

Import the existing Transfer server, or use a different endpoint type/hostname combination. Ensure Route 53 records for custom hostnames don't conflict with existing records.

Example

```
resource "aws_transfer_server" "sftp" {
  identity_provider_type = "SERVICE_MANAGED"
  protocols              = ["SFTP"]

  endpoint_type = "VPC"
  endpoint_details {
    vpc_id          = aws_vpc.main.id
    subnet_ids      = aws_subnet.private[*].id
    security_group_ids = [aws_security_group.sftp.id]
  }

  logging_role = aws_iam_role.transfer_logging.arn

  tags = {
    Name = "myapp-sftp-${var.environment}"
  }
}

# Import existing server
# terraform import aws_transfer_server.sftp s-xxxxxxxxxxxxxxxxxxxx
```

#148 Error: Error creating DataSync Task – InvalidRequestException for location ARN

Root Cause Analysis (RCA)

An AWS DataSync task references a source or destination location ARN that doesn't exist, is in the wrong region, or uses an incompatible location type combination for the task type.

Solution

Create DataSync locations before tasks and reference them directly. Ensure source and destination location types are compatible with the DataSync task type.

Example

```
resource "aws_datasync_location_s3" "source" {
  s3_bucket_arn = aws_s3_bucket.source.arn
  subdirectory = "/source-data"

  s3_config {
    bucket_access_role_arn = aws_iam_role.datasync.arn
  }
}

resource "aws_datasync_location_efs" "destination" {
  efs_file_system_arn = aws_efs_file_system.main.arn
  ec2_config {
    security_group_arns = [aws_security_group.datasync.arn]
    subnet_arn          = aws_subnet.private[0].arn
  }
  subdirectory = "/destination"
}
```

```
resource "aws_datasync_task" "migration" {
  source_location_arn = aws_datasync_location_s3.source.arn
  destination_location_arn = aws_datasync_location_efs.destination.arn
  name                 = "s3-to-efs-migration"
}
```

#149 Error: Error creating Location Service Place Index – ConflictException – resource already exists

Root Cause Analysis (RCA)

An Amazon Location Service place index, map, or tracker with the same name already exists. These resources must have unique names within an account and region.

Solution

Use unique names for Location Service resources, import existing resources, or delete and recreate. Incorporate environment names to prevent conflicts across deployments.

Example

```
resource "aws_location_place_index" "main" {
  data_source = "Esri" # or "HERE"
  index_name  = "myapp-places-${var.environment}"
  description = "Place index for ${var.environment}"

  data_source_configuration {
    intended_use = "SingleUse"
  }

  tags = local.common_tags
}

resource "aws_location_map" "main" {
  map_name = "myapp-map-${var.environment}"
  description = "Map for ${var.environment}"

  configuration {
    style = "VectorEsriNavigation"
  }
}
```

#150 Error: Error creating IoT Topic Rule – InvalidRequestException for SQL statement

Root Cause Analysis (RCA)

An AWS IoT Core topic rule contains an invalid SQL statement. The IoT SQL syntax is similar to standard SQL but has specific functions and syntax requirements for IoT data.

Solution

Validate the IoT SQL statement using the AWS IoT console's SQL validator. Ensure topic paths are correctly quoted, functions are supported by IoT SQL, and the FROM clause uses valid topic patterns.

Example

```
resource "aws_iot_topic_rule" "temperature" {
  name      = "high_temperature_alert"
  description = "Alert when temperature exceeds threshold"
  enabled   = true

  # IoT SQL syntax – note the FROM clause uses topic()
}
```

```
sql      = "SELECT temperature, deviceId, timestamp() as event_time FROM 'sensors/+/temperature' WHERE temperature > 30"
sql_version = "2016-03-23"

sns {
  message_format = "RAW"
  role_arn       = aws_iam_role.iot_rule.arn
  target_arn     = aws_sns_topic.alerts.arn
}

# Optional error action
error_action {
  cloudwatch_logs {
    log_group_name = aws_cloudwatch_log_group.iot_errors.name
    role_arn       = aws_iam_role.iot_rule.arn
  }
}
}
```